

CAPÍTULO

3

LINGUAGEM C

- 3.1 PROCESSO DE COMPILAÇÃO 31
- 3.2 ORGANIZAÇÃO DOS PROGRAMAS EM C 32
- 3.3 PADRÃO DE ESCRITA 34
- 3.4 DIRETIVAS DE PRÉ-COMPILAÇÃO 37
 - INCLUSÃO DE ARQUIVOS 37
 - DEFINIÇÃO E EXPANSÃO DE MACROS 37
 - COMPILAÇÃO CONDICIONAL 40
 - DIRETIVA PRAGMA 41
- 3.5 FUNÇÃO MAIN 43
 - COMEÇANDO COM WIRING: ARDUINO E CHIPKIT 44
- 3.6 ENTRADA E SAÍDA DE DADOS 45
- 3.7 EXERCÍCIOS 46

“C é peculiar, cheia de falhas, e um enorme sucesso.”

Dennis M. Ritchie

C (pronunciado como a letra c: /'se/) é uma linguagem de propósito geral, imperativa, com suporte à programação estruturada e recursão, possuindo variáveis de tipos definidos. O modo como foi projetada permite gerar códigos de máquina bastante eficientes, considerando as arquiteturas mais comuns de processadores. Além disso, apresenta ferramentas que facilitam o acesso a hardware. Por esses motivos, tem sido utilizada como substituta para aplicações que utilizavam a linguagem assembly.

Ela foi inicialmente desenvolvida por Dennis Ritchie, entre 1969 e 1973, na empresa AT&T Bell Labs. O primeiro grande projeto com essa linguagem, e que ajudou muito a sua popularização, foi a reimplementação do sistema operacional Unix.

Ela herdou muitas características da linguagem B, que era utilizada nos minicomputadores da época dentro dos laboratórios da Bell Labs. Por ser sucessora da linguagem B, chamaram-na simplesmente de linguagem C. A linguagem C foi padronizada pelo instituto de padrões americanos, ANSI, em 1989, e pela ISO, desde 1990, sendo conhecida respectivamente como

C89 ou C90. Uma revisão menor foi feita em 1995 pela ISO.

Em 1999 a ISO realizou mudanças na linguagem, como a possibilidade de comentários em uma única linha, melhor suporte a cálculos de ponto flutuante (IEEE754) e funções inline. A ANSI adotou esse padrão em 2000. É comum referenciar este padrão como C99.

O padrão em vigor é de 2011, sob o nome ISO/IEC 9899:2011. Este documento apresenta uma interface comum para criação de threads, um melhor suporte aos diferentes padrões de codificação de caracteres, macros para operação com números imaginários, entre outras novidades.

A tabela 3.1 apresenta o histórico do desenvolvimento dessa linguagem de programação. As linguagens BCPL e B constam na tabela por serem as linguagens que influenciaram no desenvolvimento da linguagem C.

Tabela 3.1: Histórico do desenvolvimento da linguagem C

Linguagem	Ano	Desenvolvida por
BCPL	1966	Martin Richards
B	1970	Ken Thompson, Dennis Ritchie
C	1972	Dennis Ritchie
K&R C	1978	Brain Kernighan and Dennis Ritchie
ANSI C	1989	Comitê da ANSI
ANSI/ISO C	1990	Comitê da ISO
ANSI/ISO C/NA1	1995	Comitê da ISO
C99	1999	Comitê da ISO
C11	2011	Comitê da ISO

Devido à popularidade que atingiu na época, a linguagem C influenciou várias linguagens atuais, incluindo C++, D, Go, Rust, Java, JavaScript, Limbo, LPC, C#, Objective-C, Perl, PHP, Python e Verilog. Estas linguagens possuem grande parte de suas estruturas de controle e outras características básicas muito similares à C, com uma estrutura sintática muito parecida.

Vários fabricantes e comunidades de software livre possuem ferramentas de compilação em linguagem C para quase todas as arquiteturas de processadores e sistemas operacionais existentes. Os fabricantes de processadores e microcontroladores em geral também disponibilizam ferramentas de compilação para seus chips.

Todas essas vantagens, tanto de facilidade de acesso ao hardware, quanto de disponibilidade e portabilidade, consagraram a linguagem C como a ferramenta ideal para programação em sistemas embarcados.

3.1 Processo de compilação

O processo de compilação pode ser visualizado como a tradução de um texto. Se estiver utilizando um compilador de linguagem C, a linguagem original é a C e a linguagem final é geralmente um código binário entendido pelo processador.

O arquivo de entrada contém o programa escrito pelo programador, também conhecido como código fonte. Este arquivo pode ser editado em qualquer ferramenta de edição de texto. O arquivo de saída, ou código de máquina, indica ao processador como executar os comandos que foram definidos no arquivo texto. O arquivo binário depende da arquitetura de processador utilizado. O mesmo código em C gerará arquivos diferentes para processadores diferentes.

O processo de compilação é realizado em quatro etapas que, em geral, são sequenciais: pré-compilação, compilação, assemblagem e linkagem, conforme apresentado na figura 3.1.

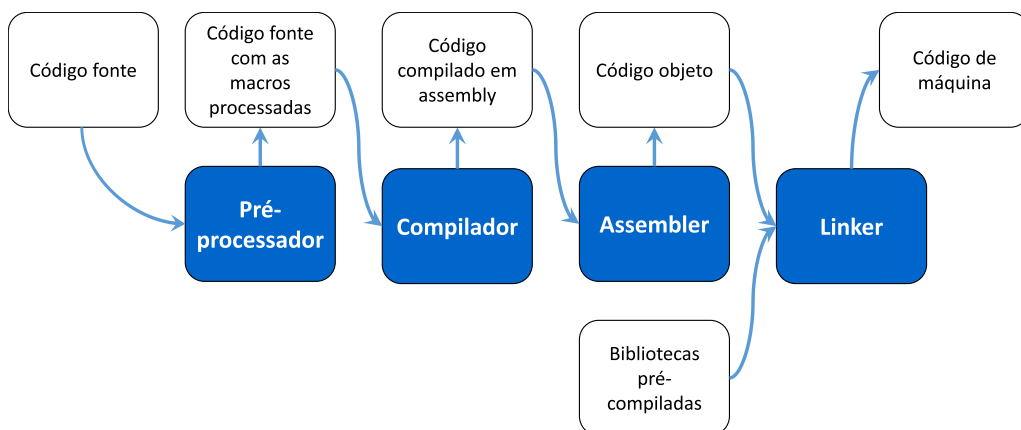


Figura 3.1: Processo de compilação de um programa em C

A etapa de pré-compilação é responsável por ler o código fonte e realizar substituições no código ou operações de configuração do compilador. Esta etapa processa os comandos incluídos no código especificamente para esta função, como `#define`, `#include`, `#ifndef`, `#endif`, `#pragma`, entre outros. Os comentários também são removidos nesta etapa. Em geral, os comandos aceitos na etapa de pré-compilação são iniciados pelo símbolo de hash “#”. Os arquivos `.c` e `.h` são processados e arquivos temporários são gerados com as mudanças necessárias.

A compilação é a etapa mais complexa. A tradução efetiva do código C para assembly é realizada nesta etapa. Assembly é uma linguagem de baixo nível que representa de modo simbólico os comandos da linguagem de máquina. No entanto, ela ainda não é utilizável pois não se encontra em formato binário, além de não estar devidamente localizada na memória do processador. Nesta etapa, também é executado o processo de otimização do código.

A otimização tem como objetivo reduzir a quantidade de código para diminuir o tamanho do arquivo final ou alterar o fluxo do programa para que este possa ser executado mais rapidamente. Em geral, esses dois objetivos são opostos, de modo que um programa menor provavelmente será mais demorado. Deve-se tomar cuidado com o processo de otimização, pois o compilador faz suposições que podem não ser verdadeiras para o ambiente embarcado, principalmente se

estiver operando com periféricos.

No fim da compilação para cada arquivo .c é gerado um arquivo intermediário em assembly. A linkagem é o processo de reunir todos os arquivos intermediários, resultantes da compilação, em um único arquivo. Neste momento, todas as referências são processadas e um arquivo final é montado contendo todo o código. As bibliotecas pré-compiladas, como as bibliotecas padrão da linguagem C (stdlib), são incorporadas estaticamente ao programa neste momento. O processo de linkagem dinâmica é pouco utilizado no ambiente embarcado, pois, geralmente existe apenas um programa que fará uso da biblioteca, não havendo necessidade de compartilhá-la dinamicamente.

A assemblagem é a última etapa. Ela é responsável por transcrever o código intermediário em assembly para linguagem de máquina, substituindo as referências relativas por referências fixas de endereçamento e posicionamento na memória. O resultado desta etapa é o arquivo binário.

Dependendo do compilador utilizado, é possível que ele gere um arquivo binário com a extensão .s19. Este arquivo contém o código de máquina, mas em formato de texto. Isto facilita o processo de transmissão do arquivo para um gravador que vai convertê-lo em binário e armazená-lo no chip, pois, além do código, ele contém valores de conferência para garantir que os dados não foram alterados na transmissão.

3.2 Organização dos programas em C

Um programa C é, na verdade, uma coleção de funções, criadas pelo próprio programador ou disponibilizadas em bibliotecas.

A maioria dos compiladores de linguagem C vem com uma grande quantidade de funções já criadas e compiladas em bibliotecas que são usadas dependendo da necessidade do programador. Parte destas bibliotecas se tornaram tão comuns que foram padronizadas pela linguagem.

Os componentes de um programa em C são:

- **Comentários:** podem e devem estar em qualquer ponto do programa. É aconselhável colocar informações logo no início do programa, tais como: nome do programa, nome do programador, período de elaboração, para que serve, se é parte de algum sistema maior, restrições de acesso por motivo de segurança, etc. Os comentários são escritos entre os delimitadores `/*` e `*/` e, desta forma, não são considerados pelo compilador. Outra forma de inserir comentários é utilizar duas barras seguidas `//`. Qualquer coisa que esteja escrita após as duas barras até o fim da linha será interpretada como comentário.
- **Diretivas de compilação:** não são instruções próprias da linguagem C. As diretivas são mensagens que o programador envia ao compilador para que alguma tarefa seja executada no momento da compilação. Deve haver uma linha inteira para cada diretiva, que é iniciada pelo caractere `#`. As diretivas mais comuns são `#include` e `#define`, usadas para especificar bibliotecas de funções a serem incorporadas na compilação, e macro substituições, respectivamente.

- **Definições globais:** normalmente são especificações de constantes, tipos e variáveis que serão válidas em todas as funções que formam o programa. Embora sejam de relativa utilidade, não é uma boa prática de programação definir muitas variáveis globais. Como elas podem ser acessadas em qualquer parte do programa, mesmo um breve descuido na alteração dos seus valores, pode provocar problemas em muitos outros locais.
- **Protótipos de funções:** são usados pelo compilador para fazer verificações durante a compilação, ver se as partes do programa que acionam as funções o fazem de modo correto, com o nome certo, com o número e tipo de parâmetros adequados, etc. Esta é uma boa prática para se programar em C.
- **Definições de funções:** são os blocos do programa onde são definidos os comandos a serem executados em cada função. A função pode, ou não, receber valores que serão manipulados em seu interior. Após o processamento, as funções podem, se necessário, retornar um valor. É obrigatório a presença de pelo menos uma função com o nome **main**, e esta será a função por onde começa a execução do programa. Não há ordem obrigatória para codificar as funções. No entanto, sempre começa pela função **main**, para facilitar a tarefa de manutenção do programa. Todas as outras funções serão codificadas em seguida.

No trecho do código 3.1, é apresentado um exemplo de programa em linguagem C com todos os componentes.

Código 3.1: Exemplo de programa em linguagem C

```
1 //comentário utilizado como cabeçalho do arquivo
2 //programa 1
3 //função: descrição
4 //autor: nome
5
6 //diretivas de compilação
7
8 //inclúdes de bibliotecas locais
9 #include "biblioteca.h"
10
11 //inclúdes de bibliotecas padrões
12 #include <stdio.h>
13
14 //definições de tipo
15 #define macros
16 #define labels
17
18 //seção de variáveis globais
19 char variavelGlobal;
20
21 //seção de protótipos de funções
22 void funcao1(char var);
23 int funcao2(void);
24
25 //função main
26 void main(void) {
27     //...
28 }
29
30 //demais funções do programa
31 void funcao1(char var) {
32     //...
33 }
34
35 int funcao2(void) {
36     int x;
37     //...
38     return x;
39 }
```

3.3 Padrão de escrita

A linguagem C é bastante flexível quanto ao modo de escrita. Existem diversas opções sobre onde abrir uma chave, por exemplo: na mesma linha do **if** ou em uma linha própria. O conjunto de escolhas é denominado padrão de escrita.

Obedecer a um padrão na hora de programar facilita a visualização do código e pode ajudar a evitar erros mais simples. Nesta seção será apresentado o padrão utilizado neste livro.

```
1 void main(void){
2 unsigned int i;
3 unsigned int newKey=0;
4 unsigned int key=0;
5 serialInit();
6 ssdInit();
7 lcdInit();
8 adInit();
9 for(;;){
10 ssdUpdate();
11 newKey = kpRead();
12 if(newkey != key){
13 newkey = key;
14 for(i=0; i<10; i++){
15 if(BitTst(key,i)){
16 lcdChar(i+'0');
17 }
18 }
19 }
20 for(i=0; i<1000; i++);
21 }
22 }
```

O primeiro ponto importante de um padrão é a indentação dos blocos. Em linguagem C, os blocos são conjuntos de códigos delimitados por chaves: “{” e “}”. Para indicar que os comandos pertencem ao mesmo código convencionou-se a adição de um espaço antes da linha, de modo a deslocar todo código interno ao bloco para a direita.

Se existir um segundo bloco dentro do primeiro, este deve ser deslocado duas vezes, indicando uma hierarquia no fluxo do programa. O trecho do código 3.2 é um exemplo de programa sem uso de nenhuma indentação.

Por não estar indentado adequadamente, fica difícil entender quais comandos são executados dentro de cada loop. A indentação resolve este problema em particular, como pode ser visto no trecho do código 3.3.

Pode-se notar, pelos códigos anteriores, que no código indentado é mais fácil visualizar quais instruções/rotinas estão subordinadas às demais.

Outra característica de padronização está na criação de nomes de funções e de variáveis. Pela linguagem C, uma função ou variável pode ter qualquer nome, desde que seja iniciada por uma letra, maiúscula ou minúscula, e os demais caracteres sejam letras, números ou underscore “_”.

A linguagem C permite que sejam declaradas duas variáveis com mesmo nome, caso possuam letras diferentes apenas quanto à caixa (maiúscula ou minúscula). Por exemplo: **var** e **Var** são variáveis distintas, o que pode gerar erro no desenvolvimento do programa, causando dúvidas e erros de digitação.

Para evitar esses problemas é comum criar uma convenção no modo de escrita das variáveis.

```
1 void main(void){
2     unsigned int i;
3     unsigned int newKey=0;
4     unsigned int key=0;
5     serialInit();
6     ssdInit();
7     lcdInit();
8     adInit();
9     for(;;){
10         ssdUpdate();
11         newKey = kpRead();
12         if(newkey != key){
13             newkey = key;
14             for(i=0; i<10; i++){
15                 if(BitTst(key,i)){
16                     lcdChar(i+'0');
17                 }
18             }
19         }
20         for(i=0; i<1000; i++);
21     }
22 }
```

A opção adotada é fazer com que os nomes de variáveis sejam escritos apenas utilizando minúsculas. Caso o nome seja composto, utiliza-se uma letra maiúscula para cada nova palavra, como nas variáveis **contaPosicao** e **contaValorTotal**. Esta abordagem é conhecida como camelCase.

Os nomes de funções podem, por exemplo, ser escritos com a primeira letra maiúscula e, no caso de nome composto, cada inicial será grafada também em maiúscula, por exemplo: **InicializaTeclado()** ou **PararSistema()**.

Os nomes utilizados nas “tags” ou “labels” de definições, utilizados na diretiva **#define**, serão grafados exclusivamente em maiúsculas como **NUMERO_DE_VOLTAS** ou **CONSTANTE_GRAVITACIONAL**. Para facilitar a leitura utilizaremos um underscore “_” entre as palavras.

Com relação à utilização das chaves, optou-se por abrir o bloco na mesma linha do comando e fechar o bloco numa linha própria, alinhando a chave com o comando que a abriu. Isto reduz a quantidade de linhas necessárias para escrever o programa. No entanto, não há nenhuma diferença prática na execução do programa, nem de velocidade ou de tamanho. Essa abordagem foi escolhida simplesmente pela economia de espaço na visualização dos códigos.

As regras apresentadas visam fornecer uma identidade visual ao código. Tais regras não são absolutas, servem apenas para o contexto deste livro. Em geral, cada instituição ou projeto possui seu próprio conjunto de normas. É importante ter conhecimento deste conjunto e aplicá-lo em seu código.

3.4 Diretivas de pré-compilação

As diretivas de pré-compilação são instruções inseridas nos arquivos de código que visam alterar o programa antes do processo de compilação. Essas instruções não existem no programa final. Todas as diretivas de compilação começam com o sinal **#**, conhecido como jogo da velha ou hash.

Inclusão de arquivos

A diretiva de compilação **#include** é a responsável por permitir que o programador utilize no seu código funções que foram implementadas em outros arquivos, seja por ele próprio ou por outras pessoas. O agrupamento de funções em um arquivo para permitir seu uso futuro é chamado de biblioteca.

Os arquivos com a extensão **.h** são denominados de arquivos de cabeçalho (arquivos “Header”) e geralmente possuem apenas os protótipos de função ou definição de tipos de variável disponibilizados pela biblioteca.

Se o código fonte for distribuído junto com a biblioteca ele possui, em geral, a extensão **.c**. Se for o caso, o programador não precisa distribuir o código fonte, podendo optar por entregar apenas o código objeto.

Deste modo, a biblioteca é geralmente composta por dois arquivos, um header e uma implementação, ou um header e um arquivo objeto.

Para fazer uso de uma biblioteca externa dentro de um arquivo, utiliza-se a diretiva **#include**. Existem dois modos de chamar esta diretiva, a diferença entre elas se encontra no modo como o compilador procura as bibliotecas:

```
1 #include <nome_arquivo.h> // arquivo do path do compilador
2 #include "nome_arquivo.h" // arquivo no mesmo diretório
```

A primeira opção, em que o nome do arquivo é escrito entre os sinais de maior e menor, indica ao compilador que ele deve procurar o arquivo citado nos diretórios conhecidos por ele. Esta opção é utilizada em geral para incluir as bibliotecas padrões da linguagem C. A segunda opção indica que o arquivo a ser incluído pode estar no mesmo diretório que o arquivo atual.

Para simplificar o desenvolvimento, é comum separar as funcionalidades em arquivos diferentes. Em sistemas embarcados, uma opção é separar por tipo de periférico, como portas digitais, comunicação serial ou um display de LCD.

Definição e expansão de macros

Outra funcionalidade do processo de pré-compilação é poder efetuar troca de símbolos, valores ou textos antes de compilar o arquivo. Isto pode ser feito através da definição de macros, pela diretiva **#define**.

O procedimento de avaliar as macros e efetuar as trocas é conhecido como expansão de macros.

A definição pode ser feita de dois modos. No primeiro, define-se um identificador e, em seguida, o texto que substituirá o identificador cada vez que ele for utilizado no código:

```
#define nome <lista de comandos>
```

Onde:

nome	É o nome (label) pelo qual a macro será conhecida ou chamada
lista de comandos	São os comandos que substituirão a macro

Um equívoco bastante comum é que a diretiva **#define** cria constantes. A utilização do **#define** permite criar identificadores ou nomes que serão substituídos pela lista de comandos. Na prática, isto permite inserir valores constantes no código de modo simplificado. No entanto, o processo de otimização do compilador pode pré-computar os valores de modo que o “valor constante” definido através deste método, nem mesmo apareça no código final. Por exemplo:

```
1 #define SEGUNDOS_MINUTOS 60
2 #define MINUTOS_HORAS 60
3 #define HORAS_DIAS 24
4
5 int segundos_para_dias(int segundos){
6     return segundos / (SEGUNDOS_MINUTOS * MINUTOS_HORAS * HORAS_DIAS);
7 }
```

Após a substituição dos valores, a função será escrita como:

```
1 int segundos_para_dias(int segundos){
2     return segundos / (60*60*24);
3 }
```

Como os três valores são constantes, durante o processo de compilação é possível que o compilador otimize esta linha para evitar recalcular o resultado, que será constante, toda vez que a função fosse executada. Deste modo, a função será reescrita como:

```
1 int segundos_para_dias(int segundos){
2     return segundos / (86400);
3 }
```

Assim, os valores iniciais das “constantes” nem mesmo aparecem na função final. Eles serviram apenas como unidades de substituição.

O segundo modo de definição permite criar macros que fazem a substituição de um texto, mas mantendo algumas informações. Essas informações que serão mantidas após a substituição são escritas em uma lista de parâmetros:

```
1 #define <nome>(<lista_parâmetros>) <lista_comandos>
```

Onde:	nome	É o nome pelo qual a macro será conhecida ou chamada.
	lista_parâmetros	São os “parâmetros” que serão enviados para a lista de comandos.
	lista_comandos	São os comandos que substituirão a macro fazendo uso dos parâmetros.

Esta diretiva é utilizada quando se deseja criar uma estrutura similar à função, mas que seja mais rápida. No entanto, existem algumas limitações. Não é possível criar variáveis locais nem fazer recursão. Além disso, o tamanho do arquivo final será maior, já que cada “chamada” de função terá o código copiado para onde a função foi chamada. Deste modo, o código aparecerá quantas vezes ele for utilizado, ao invés de uma única vez, como uma chamada de função real.

O exemplo a seguir mostra uma função com diretiva **#define** que calcula a média entre três valores:

```
1 #define Media(a, b, c) ((a+c+b)/3)
2 void main (void){
3     int x=10, y=20, z=30, resultado;
4     resultado = Media(x,y,z);
5 }
```

Depois da etapa de pré-compilação, a função será substituída e o código do programa re-escrito da seguinte maneira:

```
1 void main (void){
2     int x=10, y=20, z=30, resultado;
3     resultado = ((x,z,y)/3);
4 }
```

Ao substituir o código, o pré-compilador executa as mudanças obedecendo exatamente a estrutura definida. Por isso, ele mantém as duas chaves mesmo não precisando. Notem que as posições das variáveis z e y estão trocadas depois da substituição. Isso acontece porque na definição da função as variáveis b e c aparecem trocadas. Como o pré-compilador apenas faz a substituição dos termos envolvidos, ele obedece exatamente o que for informado pelo programador.

Por fim, existe a diretiva **#undef**, que remove a definição de uma macro, caso ela exista. Isto permite modificar o valor de uma macro que já tenha sido definida anteriormente.

Uma outra função bastante comum da diretiva **#define** é criar um conjunto de referências que serão utilizadas ao longo do programa. Isto inclui as definições de hardware do dispositivo, como endereços de periféricos ou definições padrão.

Compilação condicional

Existem 6 diretivas que permitem condicionar a compilação de um código baseada em uma premissa. São eles: **#ifdef**, **#ifndef**, **#else** e **#endif**. O funcionamento destas diretivas é muito parecido com a estrutura de decisão da programação.

```
1 #if<cond1>
2     <código mantido se cond1 = verdadeira>
3 #elif<cond2>
4     <código mantido se cond1 = falso mas cond2 = verdadeira>
5 #else
6     <código mantido se ambas condições foram falsas>
7 #endif
```

A condição pode ser escrita utilizando constantes inteiras, caracteres, operações aritméticas, binárias ou lógicas. Macros também podem ser utilizadas. Neste caso, as macros são expandidas antes de executar a operação.

```
1 int simulaQueda(int altura){
2     #if(GRAVIDADE == 9.807)
3         calculaQuedaComResistênciaAr();
4     #elif
5         calculaQuedaSemResistenciaAr();
6     #endif
7 }
```

As diretivas **#ifdef** e **#ifndef** realizam decisões baseadas no fato de uma macro ter sido definida ou não. São bastante utilizadas para gerar a configuração do sistema.

Supondo uma empresa que possui três termômetros, uma versão mais barata teria apenas um alarme sonoro para indicar a condição de temperatura alta (sobretemperatura). Uma versão intermediária teria um display de LCD mostrando o valor atual. Uma versão completa teria tanto o LCD quanto o alarme sonoro.

Ao invés de criar três programas separados, os programadores optaram por criar apenas um código que pode ser parametrizado utilizando as macros e as diretivas de pré-compilação, como pode ser visto no trecho do código 3.4.

A tabela 3.2 apresenta o efeito das duas opções de **#define** depois da etapa de pré-compilação. Como pode ser observado, apenas os códigos que estavam habilitados pelas macros permaneceram no documento final. Aqueles que não estavam habilitados foram removidos do código. As demais funções se mantiveram.

Algumas funções estão presentes em ambas as ocasiões. Pode ver que o código que inicializa o LCD só será executado se a diretiva **#define** LCD estiver presente, e o mesmo acontece com o alarme de som. Assim, o mesmo código base pode ser utilizado para todos os produtos.

Código 3.4: Exemplo de uso das diretivas condicionais de compilação

```
1 #define LCD
2 #define SOM
3
4 void printTemp(char valor){
5 #ifdef LCD
6     lcdPrint(valor);
7 #endif //LCD
8
9 #ifndef SOM
10     if(valor > 30){
11         StartAlarm();
12     } else {
13         StopAlarm();
14     }
15 #endif //SOM
16 }
17
18 void main (void){
19     int temp;
20     temperatureInit();
21
22 #ifdef LCD
23     lcdInit();
24 #endif //LCD
25
26 #ifndef SOM
27     soundInit()
28 #endif //SOM
29
30     for(;;){
31         temp = ReadTemp();
32         printTemp(temp);
33     }
34 }
```

Diretiva pragma

A diretiva **#pragma** funciona de modo bastante diferente das outras diretivas de pré-compilação. Ela não altera o código fonte, mas fornece instruções especiais ao compilador alterando o processo de compilação ou especificando detalhes extra-código.

Tabela 3.2: Comparação entre as duas opções com #define

<pre> 1 #define LCD 2 //a opção SOM não está definida 3 void printTemp(char valor){ 4 lcdNumber(valor); 5 } 6 7 void main (void){ 8 int temp; 9 temperatureInit(); 10 lcdInit(); 11 for(;;){ 12 temp = ReadTemp(); 13 printTemp(temp); 14 } 15 }</pre>	<pre> 1 //a opção LCD não está definida 2 #define SOM 3 void printTemp(char valor){ 4 if(valor > 30){ 5 StartAlarm(); 6 } else { 7 StopAlarm(); 8 } 9 } 10 11 void main (void){ 12 int temp; 13 temperatureInit(); 14 soundInit() 15 for(;;){ 16 temp = ReadTemp(); 17 printTemp(temp); 18 } 19 }</pre>
---	--

Essa diretiva depende da implementação do compilador, ou seja, o seu funcionamento, bem como as opções disponíveis, e depende do fabricante do compilador e do processador para o qual o código está sendo compilado.

#pragma <definição> <opções>	
Onde:	definição Define que tipo de comando pré-compilação será executado. opções São as opções passadas para o comando de pré-compilação.

A opção **CODE_SEG** é uma opção que pode não estar presente em todas as plataformas, ou pode assumir outros nomes. Ela indica ao linker em qual posição da memória ele deve armazenar as variáveis ou funções escritas depois do comando.

<pre> 1 #pragma CODE_SEG __NEAR_SEG NON_BANKED 2 //a variável abaixo será armazenada em memória não paginável 3 int variável_nao_paginável; 4 5 #pragma CODE_SEG DEFAULT 6 //a variável abaixo está armazenada na região padrão 7 //da memória, podendo não estar mapeada caso haja uma 8 //troca de banco de memória 9 int variável_paginável;</pre>

Outro uso desta diretiva é configurar registros específicos do microcontrolador. Em geral, estes registros controlam as configurações básicas do processador, como por exemplo, frequência de funcionamento, capacidade de debug, entre outras opções.

Para saber quais configurações são possíveis, é necessário checar o datasheet do microcontrolador e a documentação do compilador/linker.

O trecho do código a seguir configura algumas informações de funcionamento de um microcontrolador utilizando a diretiva **#pragma**:

```
1 #pragma config OSC=HS           //oscilador à cristal externo
2 #pragma config WDT=OFF          //watchdog desligado
3 #pragma config DEBUG = OFF      //desabilita debug
```

Existem ainda outras opções. Em geral são bastante específicas, sendo voltadas para cada chip, fabricante e compilador.

Função main

3.5

Todo sistema precisa ser iniciado de algum lugar. Os microcontroladores, assim que ligados, começam a executar os códigos a partir de um endereço pré-definido. Para vários processadores, é necessário fazer um conjunto de configurações iniciais para que seja possível executar o programa em linguagem C.

Entre essas configurações tem-se a alocação do ponteiro de stack, configuração da página de memória ativa ou, até mesmo, o carregamento do código do programa para a memória RAM. Este procedimento é executado logo após o sistema ser ligado. O código para essa inicialização é feito geralmente em assembly e implementado pelo próprio compilador. Depois da execução, a rotina inicial é chama a função **main()**.

A função **main()** é, portanto, a primeira função que pode executar algum código planejado pelo programador. Por ser a primeira função a ser executada em sistemas embarcados, ela possui um formato um pouco diferente das implementações tradicionais, já que ela não recebe nem retorna nada, conforme o trecho do código a seguir:

```
1 void main(void) {
2     //aqui entra o código do programa
3 }
```

Outro ponto específico deste tipo de sistema é que são projetados para começarem a funcionar assim que ligados e apenas parar quando desligados. Como todas as funcionalidades são chamadas dentro da função **main()**, espera-se que o programa continue executando as instruções dentro dela até ser desligado ou receber um comando para desligar. Esse comportamento pode ser obtido através de um loop infinito.

Para se fazer um loop infinito basta utilizar uma estrutura de repetição que sempre seja verdadeira. A seguir, estão as duas alternativas bastante utilizadas.

```
1 void main (void){
2     for(;;){
3         //aqui entra o
4         //código principal
5     }
6 }
```

```
1 void main (void){
2     while(1){
3         //aqui entra o
4         //código principal
5     }
6 }
```

Não existe diferença de implementar o loop infinito entre os dois modos apresentados.

Começando com Wiring: Arduino e Chipkit

As plataformas compatíveis com Arduino utilizam uma linguagem derivada da C++, chamada Processing, em conjunto com o framework Wiring. Nesta plataforma não se tem acesso direto à rotina **main()**. A Wiring provê, no entanto, duas funções: a **setup()** e a **loop()**.

A função **setup()** é executada uma única vez quando o sistema é iniciado. Já a função **loop()** é executada ciclicamente enquanto o sistema permanecer ligado. A função **main()** é, portanto, escondida do programador, mas pode ser encontrada dentro da pasta do Arduino. A implementação completa da função **main()** do framework Wiring é apresentada no código 3.5:

Código 3.5: Função main() do framework Wiring

```
1 int main(void){
2     init();
3     #if defined(USBCON)
4         USBDevice.attach();
5     #endif
6     setup();
7     for(;;){
8         loop();
9         if(serialEventRun) serialEventRun();
10    }
11    return 0;
12 }
```

Assim que a placa é ligada, a função **main()** é chamada. Ela executa uma inicialização básica do sistema e, se a placa faz uso de conexão USB, realiza a conexão com a USB para permitir a gravação e depuração da placa. Logo em seguida, é executada a função **setup()** que foi implementada pelo programador.

Um loop infinito é criado pela instrução **for(;;)**. Dentro dele há duas funções: a **loop()**, que executa os comandos definidos pelo programador, e **serialEventRun()**, que executa comandos de eventos da serial. Ambas as funções são de responsabilidade do programador.

Na maioria dos exemplos deste livro pode-se simplificar o funcionamento e entender a rotina de inicialização do Arduino como se ela fosse implementada conforme o trecho de código a seguir.

```
1 void main(void){
2     setup();
3     for(;;){
4         loop();
5     }
6 }
```

3.6 Entrada e saída de dados

O foco deste capítulo é a apresentação da linguagem C como uma ferramenta para a programação de sistemas embarcados. Estes sistemas, em geral, possuem poucos recursos para entrada ou saída de dados.

A placa de desenvolvimento utilizada possui um caminho de comunicação serial que pode ser utilizado para receber ou enviar informações para o computador. Neste tipo de comunicação, todas as informações trafegam como textos, sendo necessários alguns passos para converter essa informação em números ou em outros tipos de dados.

Além da comunicação serial, a placa de desenvolvimento possui um LCD e um display de 7 segmentos com quatro dígitos para exibir informações e um teclado com 10 botões para receber informações. Todos os exemplos apresentados utilizarão algum destes dispositivos de exibição.

Para que estes dispositivos funcionem corretamente é necessário inicializá-los no começo do programa. O teclado, além da inicialização, realiza um processo de leitura das teclas que deve ser constantemente executado através da função **kpDebounce()**. Por fim, para realizar a leitura de uma tecla basta utilizar a função **kpReadKey()**.

Para escrever no LCD pode-se utilizar duas funções diferentes: uma função para escrever números **lcdNumber()** e uma para textos **lcdString()**. A função **lcdPosition()**, por sua vez, posiciona o cursor segundo a linha e a coluna informada.

O trecho de código a seguir realiza a leitura das teclas do teclado e imprime no LCD, sempre na posição inicial.

```
1 void main (void){
2     int tecla;
3     kpInit();
4     lcdInit();
5     for(;;){
6         kpDebounce();
7         tecla = kpReadKey();
8         lcdPosition(0,0);
9         lcdNumber(tecla);
10    }
11 }
```

Para as plataformas Arduino e Chipkit, baseadas na Wiring, as funções de inicialização são executadas dentro da função **setup()**. As demais funções ficam dentro da função **loop()**.

```
1 void setup (void){
2     kpInit();
3     lcdInit();
4 }
5
6 void loop(void){
7     int tecla;
8     kpDebounce();
9     tecla = kpReadKey();
10    lcdPosition(0,0);
11    lcdNumber(tecla);
12 }
```

3.7 Exercícios

Ex. 3.1 — O que é uma linguagem de alto nível? Qual a sua diferença para uma linguagem de baixo nível?

Ex. 3.2 — Como é o processo de compilação de um programa?

Ex. 3.3 — Qual a função do compilador? E do linker?

Ex. 3.4 — Qual a necessidade de se criar um padrão de escrita para os programas?

Ex. 3.5 — O que é o erro de referência cíclica? Como evitá-lo?

Ex. 3.6 — Em que momento do processo de compilação as diretivas do tipo **#pragma** são executadas?

Ex. 3.7 — Quais os modos de entrada e saída disponíveis na placa de desenvolvimento? Como imprimir uma função corretamente?

Ex. 3.8 — Qual a relação entre as funções **setup()**, **loop()** e **main()**?

Ex. 3.9 — Porque em alguns sistemas embarcados a função **main()** não recebe nem retorna nenhum valor?

VARIÁVEIS

- 4.1 UTILIZAÇÃO DE NÚMEROS E SEUS TIPOS 49
- 4.2 DECLARAÇÃO DE VARIÁVEIS 51
 - INICIALIZAÇÃO DAS VARIÁVEIS 51
 - INICIALIZAÇÃO DE CONJUNTO DE CARACTERES 52
 - VÍRGULA 53
- 4.3 CONVERSÃO DE TIPOS 53
 - PROMOÇÃO DE TIPOS 54
 - PERDA DE INFORMAÇÃO NA CONVERSÃO DE TIPOS 54
- 4.4 MODIFICADORES 55
 - MODIFICADORES DE TAMANHO E SINAL 55
 - MODIFICADORES DE SINAL 56
 - MODIFICADORES DE ACESSO 56
 - MODIFICADOR DE ARMAZENAMENTO 58
- 4.5 PONTEIROS 59
- 4.6 EXERCÍCIOS 62

“A constante de um homem é a variável de outro.”

Alan Perlis

Na linguagem C todas as informações são armazenadas em estruturas denominadas variáveis. A declaração de uma variável é bastante simples:

```
<tipo> nomeDaVariavel = valorInicial;
```

Onde:

tipo	É o tipo da variável, e indica seu tamanho e modo de interpretação.
nomeDaVariavel	É um identificador para a variável criada, deve ser único.
valorInicial	É o valor que o programa dará inicialmente para a variável.

As variáveis são definidas com um nome e um tipo. Para criar o nome da variável podem ser utilizados quantos caracteres forem desejados, onde o primeiro caractere deverá ser uma

letra ou sublinhado. Importante lembrar que a linguagem C faz distinção entre maiúsculas e minúsculas. Sendo assim, **matrix** e **MaTrIx** são variáveis distintas.

Outra restrição é que a variável não pode ter o mesmo nome de uma palavra reservada em linguagem C. As palavras reservadas são:

- **auto**
 - **break**
 - **case**
 - **char**
 - **const**
 - **continue**
 - **default**
 - **do**
 - **double**
 - **else**
 - **enum**
- **extern**
 - **float**
 - **for**
 - **goto**
 - **if**
 - **int**
 - **long**
 - **register**
 - **return**
 - **short**
 - **signed**
- **sizeof**
 - **static**
 - **struct**
 - **switch**
 - **typedef**
 - **union**
 - **unsigned**
 - **void**
 - **volatile**
 - **while**

O tipo define como a variável deve ser interpretada: se ela representa um número com vírgula, se ela pode armazenar valores positivos ou negativos, se ela indica um endereço ou se representa um caractere, entre outras características. Dependendo da escolha, ela pode ocupar uma quantidade maior ou menor de memória.

Existem quatro tipos básicos de dados na linguagem C. Estes tipos, bem como a faixa de valores que conseguem armazenar, são apresentados na tabela 4.1. A faixa de valor, bem como a quantidade de bits das variáveis, são dependentes da arquitetura utilizada.

Tabela 4.1: Tipos de dado e faixa de valores

Tipo	Bits	Bytes	Faixa de valores
char	8	1	-128 a 127
int	16	2	-32.768 a 32.767
float	32	4	$3,402 \times 10^{-38}$ a $3,402 \times 10^{38}$
double	64	8	$1,797 \times 10^{-308}$ a $1,797 \times 10^{308}$

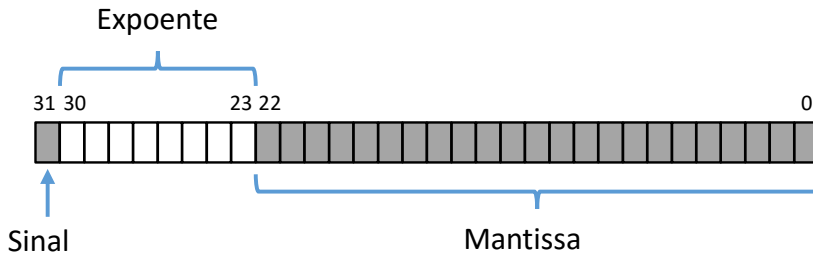
As variáveis que ocupam mais espaço na memória podem armazenar valores maiores. Assim, percebe-se também que, apenas os tipos **float** e **double** possuem casas decimais.

O tamanho do tipo **char** é definido nos padrões da linguagem C. Já o tipo **int** possui apenas uma restrição, que ele deve ser de pelo menos 16 bits. Deste modo, o tamanho real do inteiro depende do processador e do compilador utilizado. O mais comum é que ele possua o tamanho que o processador possa trabalhar naturalmente. Assim, é comum que os processadores de 64 bits possuam os inteiros com 64 bits, os de 32 bits com inteiros de 32 e os processadores de 16 bits com inteiros de 16. Os processadores de 8 bits possuem inteiros de 16 bits dada a restrição da norma.

Os tipos **float** e **double** não possuem uma definição de tamanho. A norma exige uma quantidade mínima de bits para esses dois tipos, não definindo um máximo. Para a maioria dos

compiladores é comum adotar o padrão IEEE 754, que utiliza 4 bytes para precisão simples (**float**) e 8 bytes para precisão dupla (**double**).

Estes valores conseguem representar números com vírgulas de maneira similar à que se escreve os números em notação científica: armazenando uma mantissa e um expoente. A figura 4.1 apresenta a relação entre os bits e o número com precisão simples.



Formato de precisão simples IEEE 754

Figura 4.1: Formação do número do tipo **float** segundo padrão IEEE 754

Neste caso, o valor M (mantissa) é sempre um valor entre -1 e 1. Já o valor do expoente E são números inteiros, positivos ou negativos. A mantissa representa a precisão do número, já o expoente, representa a faixa de valores que podem ser representados.

4.1 Utilização de números e seus tipos

A linguagem C trata a princípio, de todo número que estiver escrito no meio do código como um valor do tipo inteiro e sinalizado. Por padrão, os números são interpretados como escritos no formato decimal. Durante o processo de compilação, estes números são transformados em binário para serem salvos na memória do microcontrolador.

Em algumas situações, pode ser mais simples ou conveniente, escrever os números numa base diferente da decimal, seja a binária ou a hexadecimal. Isto pode ser feito utilizando os prefixos **0b**, para binário, ou **0x**, para hexadecimal, antes do número. Deve-se lembrar que, no caso de números binários, só são aceitos os dígitos zero e 1. Para os hexadecimais são aceitos os dígitos de zero a 9 e as letras de A a F. As letras podem ser escritas em maiúscula ou minúscula.

```
1 // 20010 = C816 = 110010012
2 unsigned int dec = 200;           //valor 200 escrito em decimal
3 unsigned int hex = 0xC8;          //valor 200 escrito em hexadecimal
4 unsigned int bin = 0b11001001;    //valor 200 escrito em binário
5 //todas as três variáveis possuem o mesmo valor
```

Não importa em qual base o número foi inserido, todos serão convertidos para binário no momento do armazenamento. Deste modo, todas as comparações no código exemplo são verdadeiras, já que os números 200₁₀, C₈₁₆ e 11001001₂ representam a mesma quantidade, apenas em bases diferentes.

A linguagem C ainda permite utilizar a base octal. Para isto, basta digitar o número começando com **0**.

```
1 int a = 200;           //valor 200 em decimal
2 int b = 0200;          //valor 200 em octal, representa 192 em decimal
3 int c = 0311;          //valor 311 em octal, representa 200 em decimal
4 int d = 311;           //valor 311 em decimal
```

A base octal não é muito comum nos programas atuais. Além disso, o modo de entrada dos valores pode confundir quem estiver lendo o código, visto que o programador está treinado a ignorar os zeros à esquerda do número. Por esses motivos, essa base não será utilizada ao longo deste livro.

Para valores fracionados, utiliza-se o ponto como separador decimal. A linguagem C pressupõe que todo número digitado com um ponto é do tipo **double**. Para garantir que o número digitado seja um **float** é preciso anexar a letra **f** como sufixo ao número.

```
1 float a = 3.05;        //3.05 é convertido e armazenado como float
2 double b = 3.05;       //3.05 é armazenado como double.
3
4 //por questões de precisão no armazenamento é possível
5 //que (a) possua um valor diferente de (b)
6 if(a == 3.05){
7     //a comparação é feita entre um valor float (a) e um valor double ←
7     (3.05)
8 }
9 if(a == 3.05f){
10    //a comparação é feita entre um valor float (a) e um valor float ←
10    (3.05f)
11 }
12 if(b == 3.05){
13    //a comparação é feita entre um valor double (b) e um valor double ←
13    (3.05)
14 }
```

A primeira conversão pode ser interpretada como falsa, pois na conversão do número 3.05 para

float há um arredondamento, de modo que, o número armazenado pode não ser mais igual ao valor 3.05.

A segunda comparação é verdadeira, pois o número 3.05 é convertido antes da comparação, por causa do sufixo '**f**'. Deste modo, ambos os valores sofreram o mesmo tipo de arredondamento, possuindo o mesmo valor, que pode ser diferente de 3.05.

Por fim, a terceira comparação também é verdadeira, pois ambos os termos da comparação são do tipo **double**.

4.2 Declaração de variáveis

Uma declaração de variáveis é uma instrução que resulta em uma quantidade de memória para armazenar o tipo especificado.

Uma declaração de variável consiste no nome de um tipo, seguido do nome da variável.

<tipo> variavel;		
Onde:	<tipo> variavel	Indica o tipo que será utilizado para criar a variável. É o nome da variável.

Em alguns compiladores da linguagem C é necessário que todas as variáveis sejam declaradas antes de se executar qualquer função ou código. Deste modo, não é possível construir um loop **for** criando a variável na inicialização do loop.

Se as variáveis forem declaradas depois de alguma função, o programa acusa erro, como apresentado na tabela 4.2. Coloque a criação das variáveis sempre no início da função.

Inicialização das variáveis

É possível combinar uma declaração de variável com o operador de atribuição, para que a variável tenha um valor ao mesmo tempo de sua declaração. A linguagem C não inicializa variáveis com valores padrão, deste modo, uma variável não inicializada pode conter "lixo". Como boa prática, as variáveis devem sempre ser inicializadas no momento da declaração para evitar problemas.

Na linguagem C, os caracteres podem ser escritos entre aspas simples, '**a**', e números sem aspas, **97.6**. Notar que o separador da parte inteira e fracionária é o ponto ".", e não a vírgula ",".

Exemplo:

```
1 void main(void){
2     //declaração de variáveis com valor inicial
3     char inicial = 'A';
4     int idade = 18;
5     float peso = 60.5, salario = 300;
```

```

6
7 //utilização das variáveis
8 lcdInit();
9 lcdPrint(inicial);
10 lcdPrintNum(idade);
11 for(;;){
12 }
13 }

```

Inicialização de conjunto de caracteres

Quando uma variável é declarada na linguagem C, ela não contém valor e por isso uma boa prática é inicializá-la na declaração. Para inicializar uma variável com um conjunto de caracteres, ela deve ser declarada como um vetor de caracteres.

```

1 void main(void){
2     char nome[10] = "Jose";
3     lcdInit();
4     lcdPrintString(nome);
5     for(;;){
6     }
7 }

```

Um cuidado a ser tomado com a criação e inicialização de uma cadeia de caracteres, é que o tamanho deve ser pelo menos uma unidade maior que a quantidade de caracteres.

Tabela 4.2: Problema na ordem da declaração das variáveis

<pre> 1 //este código não compila 2 void delay(void){ 3 for(int i; i<10; i++); 4 } 5 6 void main (void){ 7 kbInit(); 8 lcdInit(); 9 10 for(;;){ 11 kbDebounce(); 12 int key; 13 key = kbReadKey(); 14 lcdNumber(key); 15 delay(); 16 } 17 } </pre>	<pre> 1 //este código compila sem problemas 2 void delay(void){ 3 int i; 4 for(i; i<10; i++); 5 } 6 7 void main (void){ 8 int key; 9 kbInit(); 10 lcdInit(); 11 12 for(;;){ 13 kbDebounce(); 14 key = kbReadKey(); 15 lcdNumber(key); 16 delay(); 17 } 18 } </pre>
--	--

Vírgula

A vírgula pode ser utilizada de dois modos dentro da linguagem C. O primeiro é como operador. Neste caso, as expressões entre vírgula são processadas da esquerda para a direita. Como a vírgula é o operador com a menor prioridade na linguagem C, todas as demais operações são executadas primeiro.

O uso da vírgula como operador não é muito comum. Sua utilização mais tradicional é a separação de informações, seja na criação de uma variável ou na passagem de parâmetros. Por exemplo:

```
1 void main (void){
2     int a=4, b=7;
3     int c;
4     c = a * b;
5     lcdInit();
6     lcdNumber(c);
7     for(;;){
8     }
9 }
```

No trecho do código anterior, as variáveis **a** e **b** são criadas como sendo do tipo **int**.

Conversão de tipos

4.3

A linguagem C permite converter um valor de um tipo para um valor de outro tipo. Este processo é denominado **typecast**. Para converter um valor de um tipo para outro basta escrever entre parênteses o nome do novo tipo antes do valor original.

(<novo_tipo>) variável/valor

Onde:

<novo_tipo>

É o tipo para o qual a variável/valor será convertida. Fica entre parênteses.

variável/valor

É o valor a ser convertido. Esse valor pode vir de uma variável.

No trecho do código 4.1, é apresentado um exemplo com três conversões de tipo. Todo número digitado sem ponto é automaticamente entendido como inteiro. Se o valor está marcado com ponto decimal, na linguagem C o tipo deste valor é definido como **double**.

Na primeira conversão do código 4.1, o valor **0.05**, que é naturalmente entendido como um **double** pelo compilador, é convertido para **float** antes de ser armazenado na variável **imposto**.

A segunda conversão é feita com o valor da variável **preco**, uma variável do tipo **float**, transformando-a em um valor **double**. Nesta situação, a variável **preco** não é alterada. O

Código 4.1: Exemplo de conversão entre tipos

```
1 double calculaPreco(void){
2     int quantidade = 1;
3     double preco = 30.0;
4     float imposto = (float)0.05;
5     double preco_final = (double) preco * imposto * (double) quantidade;
6     return preco_final;
7 }
```

valor armazenado dentro da variável **preco** é copiado para uma memória temporária e então convertido para **float**. A variável original não é alterada.

Por fim, a última conversão do código 4.1 faz a leitura do valor armazenado na variável **quantidade** e o converte para o tipo **double**. Como as três grandezas estão agora em **double** o compilador executa o processo de cálculo e armazena o resultado na variável **preco_final** do tipo **double**.

Ao realizar um typecast tenha cuidado para não perder informação. Uma conversão de um tipo **float** para um tipo **double** pode fazer com que algumas casas decimais sejam ignoradas. Qualquer conversão de um tipo com mais bits para um tipo com menos bits pode causar estouro da variável.

Promoção de tipos

Em algumas situações, o compilador, percebendo que as variáveis ou os valores de uma operação são de tipos diferentes, efetua o typecast automaticamente. Esta situação é denominada promoção de tipos.

A promoção acontece apenas quando a troca do tipo não causa perda de informação. Deste modo, o compilador sempre promove o valor com menor precisão. A ordem de promoção é dada por:

char → **int** → **float** → **double**

Perda de informação na conversão de tipos

Quando se converte uma variável com maior quantidade de bits para uma variável com uma quantidade menor, os bits mais significativos são descartados. O valor 1234, por exemplo, se armazenado numa variável do tipo **unsigned char**, ignorará os bits mais significativos, fazendo com que apenas o valor 57 seja armazenado. Supondo o tipo **int** com 16 bits, a figura 4.2 representa esta perda.

É provável que o compilador gere um aviso (warning) quando este código for compilado indicando que pode haver perda de informação na atribuição. Para indicar ao compilador que isso pode acontecer e que esta é realmente a operação que se deseja fazer, então faz-se uso de um typecast, conforme o trecho do código a seguir.

```
1 unsigned int bigNum;
```

```

unsigned int  bigNum;
unsigned char num;
bigNum = 12345;
num = bigNum;
//num = 57

```

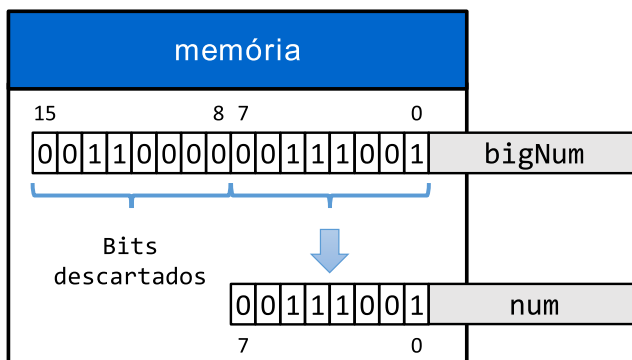


Figura 4.2: Perda de informação na conversão de um **int** para um **char**

```

2 unsigned char num;
3 bigNum = 12345;
4 num = (unsigned char)bigNum;
5 //num = 57, mas não gera um aviso (warning) pelo compilador

```

Modificadores

4.4

Modificadores são palavras reservadas que, adicionadas aos tipos básicos de dados, têm a capacidade de alterar seu comportamento, tamanho ou modo de armazenamento. Estes modificadores são inseridos na variável no momento de sua declaração e acompanham a variável por todo tempo, não sendo possível alterá-la.

```
<modificador1> ... <modificadorN> <tipo> nome;
```

Onde:

<modificadorN>

É o nome do modificador a ser utilizado.

<tipo>

É o tipo da variável, indica seu tamanho e modo de interpretação.

nomeDaVariavel

É um identificador para a variável criada, devendo ser único.

valorInicial

É o valor que o programa dará inicialmente para a variável.

A seguir serão apresentados os modificadores presentes na linguagem C.

Modificadores de tamanho e sinal

Existem dois modificadores de tamanho: **long** e **short**. Um tipo declarado com o modificador **long** pode ter tamanho *maior* ou *igual* ao tipo original. Um tipo declarado como **short** deve ter tamanho *menor* ou *igual* ao tipo original. A decisão de como os modificadores se comportam, cabe ao compilador. A tabela 4.3 mostra um exemplo para um processador onde o inteiro é de 16 bits.

Tabela 4.3: Impacto dos modificadores long e short numa variável inteira

Tipo	Bytes	Excursão máxima
int	2	-32.768 a 32.767
long int	4	-2.147.483.648 a 2.147.483.647
short int	2	-32.768 a 32.767

Como 16 bits é o menor tamanho para uma variável do tipo inteiro, o modificador **short** não faz efeito. Já o identificador **long** dobra o tamanho da variável, permitindo armazenar valores da ordem dos bilhões.

Modificadores de sinal

Existem dois modificadores de sinal: **signed** e **unsigned**. Os tipos declarados como **signed** possuem um bit reservado para o sinal. Deste modo, o valor máximo que podem atingir é menor.

Os tipos declarados como **unsigned** não podem assumir valores negativos, mas, em compensação, podem armazenar valores duas vezes maior que um tipo **signed**. A tabela 4.4 apresenta essas mudanças para as variáveis do tipo **int** e **char**.

Tabela 4.4: Impacto dos modificadores signed e unsigned nas variáveis int e char

Tipo	Bytes	Excursão máxima
unsigned char	1	0 a 255
signed char	1	-128 a 127
unsigned int	2	0 a 65.535
signed int	2	-32.768 a 32.767

Se não forem utilizados os modificadores de sinal, por padrão o tipo inteiro será sinalizado. Deste modo, não é necessário escrever **signed int**, isto já está implícito. No entanto, o tipo **char** não possui comportamento definido, podendo possuir ou não sinal, dependendo do compilador.

As variáveis do tipo **float** e **double** sempre possuem um bit de sinalização. A declaração de uma variável **unsigned double**, por exemplo, não existe.

Modificadores de acesso

Durante o processo de compilação existe uma etapa de otimização do programa. Durante esta etapa, o compilador pode retirar partes do código ou desfazer loops com períodos fixos. Por exemplo, o trecho do código a seguir:

```

1 //variável global
2 int x;
3
4 //inicio do programa
5 void main(void){
6     while(x==x);
7 }
```

Quando compilado, o loop do programa acima é transformado nos comandos em assembly como apresentado no trecho do código 4.2.

Código 4.2: Assembly para loop

```
1 //starting pCode block
2 S_Testes__main    code
3 _main:
4     .line          19 // Teste.c while (x==x);
5     BRA    _main
6     RETURN
```

Enquanto a variável **x** for igual a **x**, o programa permanece no loop. O compilador entende que esta condição sempre será verdadeira e, portanto, não há necessidade de se realizar o teste a cada rodada.

Deste modo, o compilador elimina o teste e insere a operação **BRA _main** que indica ao processador para voltar ao ponto inicial do loop. Isto acontece porque, para as variáveis comuns, seu valor só é alterado em atribuições diretas. O valor da variável não se altera sem o conhecimento do processador.

Entretanto, existe uma condição onde a variável **x** pode alterar seu valor independentemente do programa. Se esta variável representar um endereço de memória associado a um periférico físico, seu valor pode mudar independentemente do fluxo do programa. A variável **x** pode indicar o resultado de um teclado, sendo que a tecla pode ser pressionada a qualquer momento do loop. Para indicar esse tipo de situação para o compilador utilizamos a **volatile**.

```
1 //variável global
2 volatile int x;
3
4 //inicio do programa
5 void main(void){
6     while (x!=x);
7 }
```

Compilando o código fonte com a palavra **volatile**, é gerado o trecho do código 4.3.

Código 4.3: Assembly para loop com variável volatile

```
1 //starting pCode block
2 S_Testes__main    code
3 _main:
4 _00105_DS_:
5     .line          19 //teste.c while(x == x);
6     MOVLW          0x83 //primeira parte do endereço
7     MOVWF          r0x00
8     MOVLW          0x0f //segunda parte do endereço
9     MOVWF          r0x01
10    MOVFF          r0x00, FSR0L
11    MOVFF          r0x01, FSR0H
```

```

12  MOVFF    INDF0, r0x00  //realiza primeira leitura
13  MOVLW    0x83         //primeira parte do endereço
14  MOVWF    r0x01
15  MOVLW    0x0f         //segunda parte do endereço
16  MOVWF    r0x02
17  MOVFF    r0x01, FSR0L
18  MOVFF    r0x02, FSR0H
19  MOVFF    INDF0, r0x01  //realiza segunda leitura
20  MOVF     r0x00, W
21  XORWF    r0x01, W
22  BZE      _00105_DS_    //faz o teste para igualdade
23  RETURN

```

Pode ser observado que, deste modo, o compilador é forçado a ler a variável **x** duas vezes. Após armazenar as duas leituras, ele realiza o teste para ver se ela permanece com o mesmo valor. A palavra **volatile** força, portanto, que o processador realize as operações na variável sem otimizações. Esta estrutura pode ser utilizada, por exemplo, para aguardar que um evento aconteça. O programa ficará travado no loop enquanto as leituras forem iguais. Se a primeira leitura for diferente da segunda, isto significa que o evento acabou de acontecer, e que, portanto, o programa pode continuar.

Em algumas situações é necessário indicar que algumas variáveis não podem receber valores pelo programa. Para isto, é necessário usar a palavra reservada **const**. Este modificador é usado para indicar que a variável representa um local que apenas pode ser lido e não modificado, como, por exemplo, uma porta para entrada de dados. Nesta situação é comum utilizar as palavras **volatile** e **const** juntas.

```

1  volatile const int X;
2  //início do programa
3  void main(void){
4      while(x == 3){
5          //realiza tarefa enquanto a variável não muda
6      }
7  }

```

Ao tentar modificar o valor da variável **x** em algum momento do código, o compilador retornará sem sucesso à compilação com o seguinte erro:

```
main.c: error 33: Attempt to assign value to a constant variable (=)
```

Modificador de armazenamento

As variáveis em linguagem C possuem um tempo de vida que é condizente com o ambiente em que ela foi criada. Por exemplo, uma variável declarada dentro de uma função. Quando a função começa a ser executada ela reserva um espaço de memória para a variável em questão. Quando a função termina o espaço de memória que foi reservado para a variável é liberado automaticamente, fazendo com que o valor que estava armazenado na variável se perca. Estas variáveis são armazenadas na pilha de dados, conhecida como **stack**.

Já algumas variáveis possuem um espaço reservado de modo permanente. Durante todo o tempo em que a placa estiver ligada aquele espaço será utilizado exclusivamente para a variável.

Em geral, é trabalho do compilador definir onde a variável será armazenada. Para forçar uma variável a possuir um espaço reservado pode ser utilizado o modificador de armazenamento **static**. Deste modo, as variáveis locais de uma função passam a armazenar o valor, mesmo que a função termine, pois ela possui um endereço exclusivo para armazenar seu valor.

```
1 //cria um contador persistente que é
2 //incrementado a cada chamada de função
3
4 int contadorPersistente(int reseta) {
5     static char variavel_persistente;
6     if(reseta){
7         variavel_persistente = 0;
8     }else{
9         return (variavel_persistente++);
10    }
11    return -1;
12 }
```

Outro modificador de armazenamento é o **register**. Ele indica ao compilador que se deseja que a variável seja armazenada, se possível, nos registros do processador. Armazenar a variável em um registro do processador aumenta muito a velocidade com que as operações realizadas com ela acontecem. No entanto, a quantidade de registros é muito limitada. Algumas arquiteturas de processador possuem um único registro.

Outro problema com o modificador **register** é que se a variável for efetivamente armazenada no registro, pode ser impossível recuperar o endereço dela para realizar qualquer operação com ponteiros, por exemplo.

Por fim, o modificador **register** é apenas uma sugestão para o compilador. Segundo a norma, o compilador não é obrigado a obedecer esta exigência do programador. Este modificador acaba funcionando como sugestão.

O último modificador de acesso é o **extern**. Esse modificador indica que a variável em questão já foi declarada em outro lugar, geralmente em outro arquivo. Este modificador visa permitir que diferentes arquivos acessem uma mesma variável. Toda variável em linguagem C declarada fora de uma função possui o modificador **extern** implícito.

4.5 Ponteiros

Toda variável criada é armazenada em uma posição da memória. O número que identifica essa posição é chamado de endereço.

O endereço em que uma variável será salva depende de alguns fatores. Variáveis locais são, em geral, armazenadas na stack. Estas variáveis possuem endereços diferentes a cada execução da função, a menos que usem o modificador **static**.

Variáveis globais, declaradas fora das funções, possuem endereço fixo durante todo o tempo em que a placa permanecer ligada.

Para obter o endereço de uma variável o operador **&** pode ser utilizado. O valor do endereço pode então ser utilizado como um valor normal.

Conhecer o endereço de uma variável é fundamental para a criação de um ponteiro para ela.

```
1 //cria a variável a num endereço de memória a ser
2 //decidido pelo compilador
3 int a = 0;
4 a = a + 1;
5 lcdNumber( a ); //imprime o valor 1
6 lcdNumber( &a ); //imprime o endereço de a (por exemplo 17821)
```

O ponteiro é uma variável que, geralmente, possui o mesmo tamanho de uma variável do tipo inteiro. Sendo assim, obedece as mesmas faixas de valores.

A diferença entre um ponteiro e um inteiro é o significado do valor armazenado. No inteiro o valor armazenado representa um número, uma quantidade que será utilizada para algum cálculo. Já no ponteiro o valor armazenado representa uma posição de endereço na memória.

Através do ponteiro é possível manipular o conteúdo de outra variável. A variável que será manipulada é definida pelo valor que está dentro do ponteiro. Para definir um ponteiro é preciso indicar ao compilador um tipo. Neste caso, o tipo não define o tamanho do ponteiro, mas sim o tamanho da variável que o ponteiro apontará.

A sintaxe para a declaração de um ponteiro é dada por:

<code><tipo> *nomeDaVariável;</code>					
Onde:	<table><tr><td><code><tipo></code></td><td>Indica o tipo de variável que será apontado pelo ponteiro.</td></tr><tr><td><code>nomeDaVariável</code></td><td>É um identificador para a variável criada, deve ser único.</td></tr></table>	<code><tipo></code>	Indica o tipo de variável que será apontado pelo ponteiro.	<code>nomeDaVariável</code>	É um identificador para a variável criada, deve ser único.
<code><tipo></code>	Indica o tipo de variável que será apontado pelo ponteiro.				
<code>nomeDaVariável</code>	É um identificador para a variável criada, deve ser único.				

Por exemplo, para se criar dois ponteiros, um apontando para um endereço onde existe um valor inteiro e outro para um endereço onde existe um valor float, pode ser feita a seguinte declaração:

```
1 int *ptrInt;  
2 float *ptrFloat;
```

Deve-se tomar cuidado pois neste exemplo **ptrInt** e **ptrFloat** são variáveis que armazenam endereços de memória, e não os valores do tipo **int** ou **float**. A região de memória apontada pela variável **ptrInt** é que armazena um inteiro, do mesmo modo que a região de memória apontada por **ptrFloat** armazena um valor fracionário.

Assim, o ponteiro possui dois modos distintos de uso. Pode manipular o valor da variável apontada pelo ponteiro ou pode manipular o valor do próprio ponteiro. Isto significa manipular o endereço que o ponteiro armazena.

Se quiser manipular o valor do endereço, é usado o nome do ponteiro normalmente, mas, se manipular o valor que está sendo apontado por ele, então é utilizado um asterisco antes do nome da variável.

```
1 ptrInt = 1024;  
2 *ptrInt = 254;
```

A primeira instrução indica ao compilador que o ponteiro **ptrInt** receberá o valor 1024. Isto vai modificar o valor do próprio ponteiro. Depois dessa instrução o ponteiro passa a apontar para o endereço de memória 1024.

A segunda instrução armazena o valor 254 na região de memória apontada pelo ponteiro **ptrInt**. Como o ponteiro apontava para o endereço 1024, o valor 254 será armazenado lá. O valor 1024 dentro do ponteiro não será modificado pois, ele será utilizado apenas para servir de indicador.

Para trabalhar com ponteiros é preciso ter cuidado. Toda variável em linguagem C, ao ser criada, já possui um valor. Este valor pode ser resultado de operações antigas, sendo comumente chamado de lixo.

Ao ser definido, um ponteiro possui lixo como valor de endereço. Se o ponteiro for usado sem armazenar um valor correto, corre-se o risco de manipular um valor que seja importante para o funcionamento do sistema e que, a princípio, não poderia ser alterado. Neste caso, pode-se

provocar danos aos dados, ao código do programa, ou mesmo travar a placa.

É necessário tomar cuidado ao inicializar os ponteiros. O valor atribuído a eles deve ser de um endereço disponível na memória.

Um exemplo bastante comum é criar um ponteiro que aponta para o endereço de uma variável já definida.

```
1 //definindo a variável ivar
2 int ivar;
3
4 //definindo o ponteiro iptr
5 int *iptr;
6
7 //o ponteiro iptr recebe o valor do endereço da variável ivar
8 iptr = &ivar;
9
10 // as próximas linhas são equivalentes
11 ivar = 421;
12 *iptr = 421;
```

Com sistemas embarcados, existem alguns endereços de memória que possuem características especiais. Estes endereços possuem registros de configuração, interfaces com o meio externo e variáveis importantes para o projetista. Eles são definidos no momento em que o microprocessador ou microcontrolador é projetado, portanto são fixos e dependentes da arquitetura utilizada. Através dos ponteiros é possível acessar esses endereços de maneira bastante simples, mesmo em linguagem C.

Exercícios

4.6

Ex. 4.1 — Quais são os tipos de dados básicos que podem armazenar valor na linguagem C?

Ex. 4.2 — Como os tipos das variáveis podem ser modificados?

Ex. 4.3 — Qual o impacto de uma variável com o modificador volatile?

Ex. 4.4 — É possível criar uma variável com os modificadores volatile e static ao mesmo tempo? Qual é o comportamento dessa variável?

Ex. 4.5 — Assinale os nomes de variáveis válidos na linguagem C:

- () valor
- () salario-liquido
- () _b248
- () (x2)
- () nota*do*aluno
- () a1b2c3
- () 3 x 4

```
(    ) Maria
(    ) km/h
(    ) xyz
(    ) nome empresa
(    ) sala_215
(    ) "nota"
(    ) ah!
(    ) m{a}
```

Ex. 4.6 — Dê o tipo de cada um dos valores:

```
(          ) 613
(          ) -3.012 x 10
(          ) 613.0
(          ) "verdadeiro"
(          ) -613
(          ) 28.3 x 10-33
(          ) "613"
(          ) "constante"
(          ) falso
(          ) 21
(          ) 0,21
(          ) 17 x 1012
(          ) 15f
(          ) 613u
```

Ex. 4.7 — Supondo que as variáveis NB, NA, NMAT e SX sejam utilizadas para armazenar a nota do aluno, o nome do aluno, o número da matrícula e o sexo, declare-as corretamente, associando o tipo adequado ao dado que será armazenado.

Ex. 4.8 — Sendo P, Q, R e S variáveis cujos conteúdos são iguais a 2, 3, 12 e 4.5, respectivamente, quais os valores fornecidos pelas expressões aritméticas abaixo?

```
100 * Q / P + R      = (          )
P + R % 5 - Q / 2    = (          )
R % (P+1) - Q * R    = (          )
1 + (R + Q) / Q      = (          )
2 * S % 3 + S        = (          )
((20 / 3) / 3) + 2   = (          )
1 + (P*3+2*R)*(1/5) = (          )
2 + (R*10) / (1/S)  = (          )
```

Ex. 4.9 — Sendo **soma**, **num** e **X** variáveis numéricas (**int**); **nome**, **cor**, **dia**, **tudo** variáveis literais (**char[]**), assinale os comandos de atribuição inválidos:

```
(    ) num = 5;
```

```
(    ) soma = num + 2 * X;
(    ) tudo = soma;
(    ) cor = "Azul" - X;
(    ) X = X + 1;
(    ) nome = "*ABC*";
(    ) dia = "Segunda";
(    ) soma + 2 = X * 2 - num;
```

Ex. 4.10 — Quais os valores armazenados em **soma** e **nome**, supondo que **num**, **x** e **dia** valem, respectivamente, 5, 2.5, “Azul”, “Terça”?

```
nome = dia;
soma = num * 2 / x + (1/x);
```

Ex. 4.11 — Dê o valor da variável **resultado** após a execução da seguinte sequência de operações (suponha que todas as variáveis sejam do tipo inteiro):

```
resultado = 3.0 * 6;
X = 2.0;
Y = 3.0;
resultado = X * Y - X;
resultado = 4;
X = 2;
resultado = resultado * X;
```

Ex. 4.12 — Suponha que **A**, **B**, **C**, **I**, **J** e **K** sejam variáveis do tipo **double**. Dados **A** = 4.0, **B** = 6.0 e **I** = 3, qual seria o valor final dos seguintes comandos de atribuição?

```
C = A * B - I;
K = I / 4 * 6;
C = B / A + 1.5;
```

Ex. 4.13 — Implemente um programa que converta libras para newtons, sendo que há 4.9 newtons em uma libra.

Ex. 4.14 — Implemente um programa que converta pés para metros, sendo que há 3.28 pés em um metro.

Ex. 4.15 — Implemente um programa que converta milhas para quilômetros, sendo que há 1.61 quilômetros em uma milha.

Ex. 4.16 — Implemente um programa que, dado um número do tipo **double**, consiga separar a parte inteira da parte fracionária. Exemplo: número 237.48, Parte inteira: 237, Parte fracionária: 0.48

Ex. 4.17 — Implemente um programa em C que, dados dois números inteiros, imprima:

- raiz quadrada da soma dos números;
- resto da divisão do primeiro pelo segundo.

Ex. 4.18 — Implemente um programa que calcula a média aritmética de todos os números entre 13 e 25.

Ex. 4.19 — Uma revendedora de carros usados paga a seus funcionários vendedores um salário fixo por mês, uma comissão, também fixa, para cada carro vendido e mais 5% do valor das vendas por eles efetuadas. Implemente um programa que calcule o salário de um funcionário que vendeu 5 carros de R\$19.999,00, 2 de R\$28.750,00 e 1 de R\$46.700,00. O salário fixo é de R\$1.500,00 e a comissão fixa é de R\$99,95.

Ex. 4.20 — Implemente um programa para converter uma temperatura de graus Celsius para graus Fahrenheit, sendo que: $C = (F - 32)/18$

Ex. 4.21 — Implemente um programa que calcule quantas horas, minutos e segundos existem no intervalo de 67.702.556 segundos.

Ex. 4.22 — Implemente um algoritmo que, para uma determinada quantia em reais, calcule o menor número possível de notas de 100, 50, 10, 5 ou 1 em que o valor pode ser decomposto.

ESTRUTURAS COMPOSTAS

5.1	ESTRUTURAS HOMOGÊNEAS	68
	VETOR DE CHAR X STRINGS	71
	MATRIZES	72
5.2	ESTRUTURAS HETEROGÊNAS	74
5.3	BIT FIELDS	76
5.4	ENUMERADORES	78
5.5	DEFINIÇÕES DE TIPO	78
5.6	EXERCÍCIOS	80

“Parece que alcançamos os limites do que é possível alcançar com a tecnologia do computador, embora se deva ter cuidado com tal declaração, já que elas tendem a soar muito tolas em 5 anos.”

John von Neumann

5.1 Estruturas homogêneas

Um **vetor** é uma sequência de informações do mesmo tipo. As informações são chamadas elementos do vetor e são numeradas consecutivamente a partir do zero (0, 1, 2, 3, ...). Os números dos elementos do vetor são denominados valores índices. Os números localizam a posição do elemento dentro do vetor, possibilitando acesso direto ao vetor.

As variáveis que serão utilizadas para montagem do vetor podem ser de qualquer tipo de dado da linguagem C, incluindo estruturas definidas pelo usuário. A única restrição é que todas as posições possuam o mesmo tipo de dado. Estas variáveis são criadas automaticamente de forma estática quando o vetor é declarado.

Se o nome do vetor é **v**, então **v[0]** é a forma de acessar o elemento que está na posição 0, **v[1]** é a forma de acessar o elemento que está na posição 1 e assim por diante.

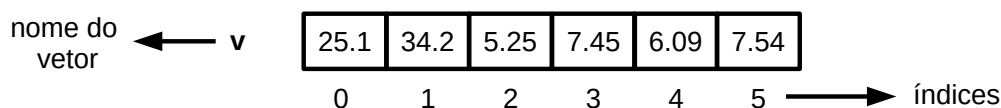


Figura 5.1: Exemplo de variável do tipo vetor

Assim como qualquer tipo de variável, deve-se declarar um vetor antes de utilizá-lo. A declaração do vetor é similar à declaração de outros tipos de dados. A única diferença é que deve ser informado a quantidade de elementos do vetor. Essa quantidade é também chamada de tamanho ou comprimento.

Para indicar o tamanho do vetor utiliza-se números positivos e inteiros entre colchetes, após o nome do vetor. A sintaxe para declaração de um vetor é:

```
<tipo> nomeVetor[numeroDeElementos];
```

Onde:

tipo	É o tipo do dado de cada posição do vetor.
nomeVetor	É um identificador para o vetor criado, deve ser único.
numeroDeElementos	É a quantidade de elementos que o vetor consegue armazenar.

O tamanho do vetor deve ser uma constante inteira positiva. Não se pode utilizar outras variáveis como indicativo do tamanho do vetor. É possível, no entanto, fazer uso de macros definidas com a diretiva **#define**.

Por exemplo, um vetor de 10 elementos do tipo **int** pode ser declarado como apresentado na figura 5.2.

Essa declaração indica ao compilador que ele deve reservar espaço suficiente na memória para armazenar 10 elementos do tipo inteiro. Além do espaço de cada um dos elementos, o compilador também reserva espaço para um ponteiro que apontará para o primeiro elemento do vetor.

Grande parte da utilidade de um vetor provém do fato de ser possível acessar os elementos do vetor de forma individual, utilizando números para indicar a posição ou até mesmo variáveis inteiras. A figura 5.3 apresenta este conceito.

Os elementos de um mesmo vetor são sempre armazenados em blocos contínuos na memória, de modo que o elemento **1** está localizado imediatamente depois do elemento **0** e antes do elemento **2**.

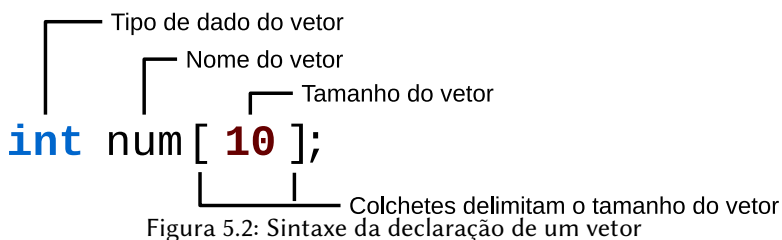
O trecho do código a seguir cria dois vetores. O vetor **num** consegue armazenar até cinco elementos do tipo inteiro. O vetor **codigo** armazena até cinco elementos do tipo **char**.

```
1 int num[5];
2 char codigo[5];
```

Um exemplo do posicionamento destes vetores na memória pode ser pensado como o mostrado na figura 5.4.

Como todas as variáveis, cada uma das posições dos vetores começam com valores não definidos. Deste modo, para evitar fazer alguma conta ou processamento com esses valores sem significado é importante atribuir valores a cada elemento do vetor. Isto pode ser feito da seguinte maneira:

```
1 nomeVetor[posicao] = novoValor;
```



```
int idade[6];
```

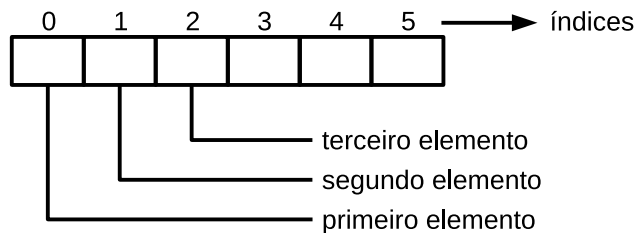


Figura 5.3: Acesso a elementos do vetor

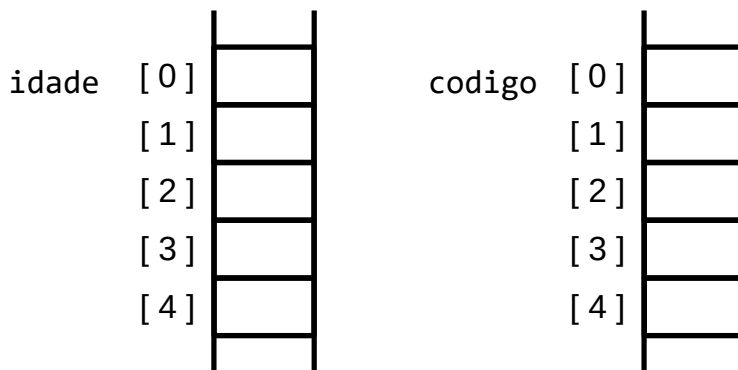


Figura 5.4: Posicionamento dos vetores na memória

A inicialização das posições do vetor `idade` pode ser escrita como no trecho de código a seguir:

```
1 num[0] = 10;  
2 num[1] = 20;  
3 num[2] = 30;  
4 num[3] = 40;  
5 num[4] = 50;
```

A primeira sentença atribui o valor **10** à variável `num[0]` e o valor **20** à variável `num[1]`. Este método não é eficaz quando o vetor contém muitos elementos.

Outro modo de se inicializar os valores de um vetor é durante sua definição. Deste modo, é possível passar todos os valores para o vetor em uma única linha de código, quando o vetor tem um tamanho pequeno. A lista de valores são escritos entre chaves e separados por vírgulas, como apresentado na figura 5.5.

Quando os elementos do vetor são inicializados deste modo, não é preciso indicar o tamanho do vetor. O compilador contará o número de variáveis que foram digitadas entre as chaves e criará um vetor do tamanho adequado.

Exemplos:

```
1 //vetor de 5 elementos  
2 int num[5] = {10, 20, 30, 40, 50};
```

```

3 //vetor de 3 elementos
4 int n[ ] = {3, 4, 5};
5 //vetor de 4 elementos
6 float k[ ] = {2.5, 2.1, 7.5, 5.6};

```

Vetor de char x Strings

O vetor de **char** é um tipo muito utilizado para armazenar um texto. Nesta situação cada letra do texto será armazenada em uma posição do vetor.

Um problema com esse tipo de vetor é saber quanto o texto termina. A convenção mais utilizada é inserir o código `'\0'` como delimitador de fim de texto. Esse código é representado pelo número 0 (zero) na codificação ASCII, sendo também conhecido como terminador.

O tamanho de posições em um vetor de **char** que armazenará uma string deve ser uma unidade maior que a quantidade de letras que se deseja armazenar, sendo que a última posição possui o elemento `'\0'`.

A linguagem C define um modelo de inicialização para permitir que os textos sejam facilmente criados e armazenados em um vetor de **char**. Para isso, basta criar um vetor de **char** com o tamanho adequado e escrever o texto entre aspas duplas.

```
char nome[tamanho] = "texto";
```

Onde:

nome	É um identificador para o vetor criado, deve ser único.
tamanho	Indica a quantidade de elementos do vetor. O tamanho deve ser uma unidade maior que a quantidade de letras.
texto	É o texto a ser armazenado no vetor. Se o tamanho for omitido na declaração, o compilador utiliza esse texto para definir o tamanho.

Por estar entre aspas duplas, o compilador automaticamente insere o terminador no final do vetor. No trecho de código abaixo, por exemplo, existem 6 letras e 1 espaço, totalizando 7 caracteres. Inserindo o terminador `'\0'`, o tamanho final do vetor **msg** é de 8 caracteres.

```
1 char msg[] = "Bom dia"; //msg possui tamanho 8
```

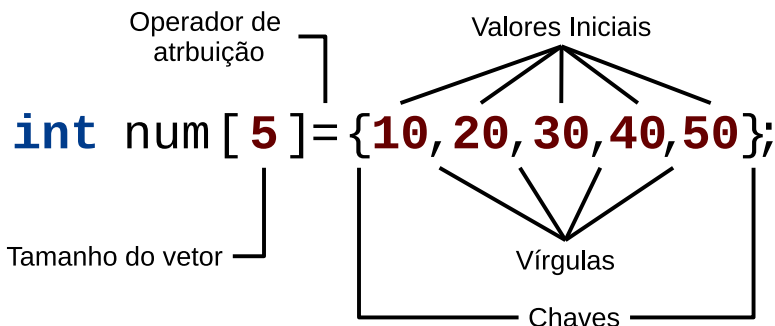


Figura 5.5: Inicialização de vetores

Matrizes

As matrizes são variáveis muito semelhantes aos vetores, possuindo um conjunto de elementos do mesmo tipo armazenados sequencialmente na memória. A diferença é que as matrizes podem possuir mais de um índice. Cada índice controla uma dimensão da matriz. Os vetores podem ser entendidos então como matrizes unidimensionais, pois possuem apenas um índice e sua representação gráfica acontece ao longo de uma linha.

As matrizes bidimensionais se assemelham a tabelas, de modo que se pode interpretar os índices como suas linhas e colunas. Este formato de tabela é apenas uma interpretação, pois, na memória, os dados são alocados sequencialmente, do mesmo modo que o vetor.

A declaração da matriz segue o mesmo formato da declaração de um vetor, com números inteiros positivos, um para cada dimensão. É necessário tomar cuidado, pois uma simples matriz de 10 linhas por 10 colunas de números flutuantes consome 400 bytes, o que pode representar quase 25% da memória de um microcontrolador mais simples.

Por exemplo, uma variável do tipo matriz de duas dimensões pode ser declarada como:

```
<tipo> nomeMatriz[tamanhoL][tamanhoC];
```

Onde:	tipo	É o tipo de dado de cada posição da matriz.
	nome	É um identificador para o vetor criado, deve ser único.
	tamanhoL	Indica o tamanho da primeira dimensão da matriz (número de linhas).
	tamanhoC	Indica o tamanho da segunda dimensão da matriz (número de colunas).

Da mesma maneira como ocorre com os vetores, os índices começam sempre em 0 (zero).

No exemplo a seguir, o trecho de código cria uma matriz chamada **a** contendo duas linhas (zero e 1) e 6 colunas (zero a 5), capaz de armazenar até dez números inteiros.

```
1 int a[2][6];
```

A figura 5.6, mostra a representação de modo gráfico, como uma tabela de duas linhas e seis colunas.

		0	1	2	3	4	5
a	0						
	1						

Figura 5.6: Matriz 2x6

Para armazenar valores na matriz basta indicar a linha e coluna em que o valor será salvo.

```
1 a[1][4] = 10;
```

A figura 5.7 apresenta o estado da matriz após a inserção do valor 10 na linha 1, coluna 4.

		0	1	2	3	4	5
a	0						
	1					10	

Figura 5.7: Atribuindo valores à matriz

As posições das matrizes que não forem inicializadas não estão vazias; na verdade, elas possuem valores que são resultado de operações anteriores. Estes valores não tem nenhum significado, sendo comumente chamados de lixo.

As matrizes podem ser inicializadas quando são declaradas. A inicialização consta de uma lista de constantes separadas por vírgulas do mesmo modo que na inicialização dos vetores.

Para efeitos de organização visual, pode-se separar cada linha com um par de chaves extras, inclusive colocando-as em linhas diferentes separadas por vírgulas.

Este último método permite apresentar os dados na inicialização com uma apresentação gráfica muito similar a uma tabela, facilitando a visualização dos dados:

```
1 //inicialização por lista contínua
2 int mat[2][3] = {51, 52, 53, 54, 55, 56};
3 //inicialização por linhas
4 int mat[2][3] = {{51, 52, 53}, {54, 55, 56}};
5 //inicialização por linhas e com quebra de linha
6 int mat[2][3] = {{51, 52, 53},
7                  {54, 55, 56}};
```

Após a definição da variável e inicialização dos dados, a matriz estará armazenada na memória conforme apresentado na figura 5.8.

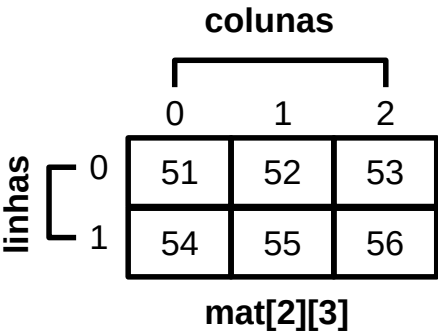


Figura 5.8: Matriz 2x3 inicializada

5.2 Estruturas heterogêneas

Os vetores são estruturas de dados que contêm um número definido de elementos e todos eles possuem o mesmo tipo de dado. O fato de todos os elementos terem o mesmo tipo de dado é uma limitação quando é necessário armazenar um conjunto de informações com elementos de tipos diferentes de dados.

A solução para esse problema é utilizar um tipo de dado chamado estrutura (struct). Os elementos individuais de uma estrutura são chamados de membros. Cada membro (elemento) de uma estrutura pode conter dados de um tipo diferente dos outros membros.

Uma estrutura pode conter qualquer número de membros. Cada membro tem um nome que serve de identificador. Este nome deve ser único. Ao contrário dos tipos básicos, é necessário de duas etapas para poder criar uma estrutura.

A primeira é declarar o formato da estrutura. Isto é feito através da palavra reservada **struct**.

<pre> struct nomeEstrutura{ tipoDadoMembro_1 nomeMembro_1; tipoDadoMembro_2 nomeMembro_2; //... tipoDadoMembro_n nomeMembro_n; }; //end struct </pre>	
Onde:	nomeEstrutura É nome dado ao formato de estrutura recém-criado. tipoDadoMembro_n tipo da variável n (int, char, float, etc). nomeMembro_n nome da variável n.

Após definir o formato, pode-se criar variáveis com esta estrutura através da palavra reservada **struct**.

<pre> struct nomeEstrutura nomeVar; </pre>	
Onde:	nomeEstrutura Nome da estrutura utilizada. nomeVar Nome da variável criada.

Por exemplo, uma estrutura para armazenar os dados de uma coleção de séries de TV. As informações relevantes para essa estrutura podem ser elencadas como os dados a seguir:

Nome do Membro	Tipo de dado
Título	vetor de tamanho 20
Produção	vetor de tamanho 30
Temporada	inteiro
Preço	ponto flutuante (float)
Data da compra	vetor de tamanho 8

O formato desta estrutura pode ser transcrito para o trecho do código utilizando os seguintes comandos:

```
1 struct serieTv{
2     char titulo[30];
3     char producao[20];
4     int temporada;
5     float preco;
6     char dataCompra[8];
7 };//end struct
```

Bit fields

5.3

Uma necessidade comum em sistemas embarcados é a capacidade de manipular apenas uma certa quantidade de bits ou até mesmo um único bit de modo individual. Isto pode ser feito através das operações bitwise ou da criação de estruturas com variáveis cujo tamanho seja determinado pelo programador. Caso a variável criada possua uma quantidade menor de bits que a de uma variável normal, bits de “padding” serão adicionados.

Se duas variáveis forem criadas dentro de uma estrutura, elas serão alinhadas em um mesmo espaço de memória, permitindo que o programador acione cada bit, ou região de bits. Esta técnica permite que bits sejam agrupados de acordo com os registros do microcontrolador.

Na figura 5.9 é apresentada a criação de duas variáveis. A primeira, **nibble**, cria uma estrutura composta de duas subseções de 4 bits cada. A segunda, **bits**, cria uma estrutura cujos dois primeiros bits podem ser acessados individualmente. Os demais bits não são utilizados e não podem ser acessados através da estrutura.

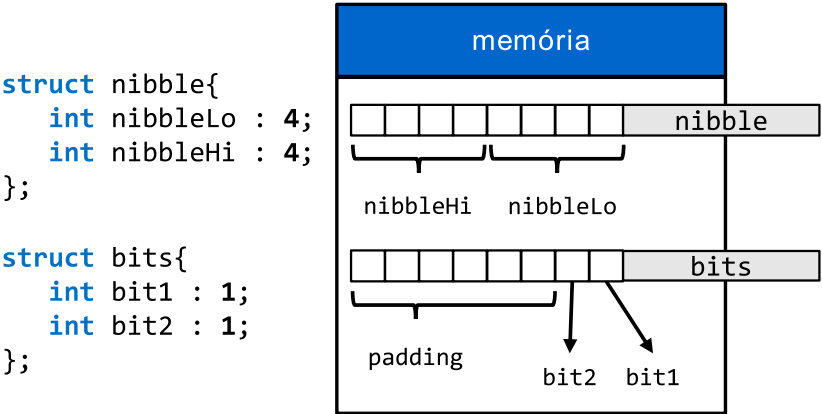


Figura 5.9: Criação de estruturas com bit fields

A sintaxe para a criação de uma estrutura com bit fields é dada por:


```

struct{
    tipo1 <nome_elemento1> : tamanho1;
    tipo2 <nome_elemento2> : tamanho2;
    tipo3 <nome_elemento2> : tamanho3;
    tipo4 <nome_elemento2> : tamanho4;
    //...
    tipoN <nome_elementoN> : tamanhoN;
};

```

Onde:

tipo	Tipo base do elemento char ou int , com ou sem os modificadores de sinal.
<nome_elemento>	Nome do elemento para posterior acesso pela estrutura.
tamanho	Quantidade de bits reservados para o elemento.

O acesso a estas estruturas é feito do mesmo modo que as estruturas tradicionais. A diferença é que, como as variáveis possuem tamanho menor, é mais fácil acontecer um estouro, ou overflow.

```

1 struct nibble{
2     unsigned int nibbleLo : 4;
3     unsigned int nibbleHi : 4;
4 };
5
6 //criação de uma variável teste1 do tipo struct nibble
7 struct nibble teste1;
8
9 //como o nibbleLo possui apenas 4 bits, só pode contar até 15 = (2^4-1)
10 teste1.nibbleLo = 15;
11
12 teste1.nibbleLo++;
13 //como a variável estourou seu valor voltou a zero. No entanto a ↵
   variável nibbleHi não será alterada
14
15 //o LCD vai imprimir o número zero
16 lcdNumber(teste1.nibbleLo);

```

Enumeradores

Enumeradores são definições da linguagem C que visam facilitar a criação de referências textuais.

```
enum{LABEL1 = VALOR1,
      LABEL2 = VALOR2,
      //...
      LABELN = VALORN};
```

Onde:

LABEL	Nome textual que será utilizado para referência do valor.
VALOR	Especificação do valor numérico do label, é opcional.

Caso os valores sejam omitidos, a estrutura automaticamente incrementa uma unidade para cada elemento do **enum**. O trecho de código abaixo apresenta um exemplo de enumerador e sua correspondência com um conjunto de **#defines**.

```
1 enum{AZUL = 1,
2     VERDE, //como não foi definido valor, VERDE = AZUL+1
3     VERMELHO = 4};
4
5 //alternativa com defines
6 #define AZUL 1
7 #define VERDE 2
8 #define VERMELHO 4
```

5.5 Definições de tipo

Os tipos básicos da linguagem C nem sempre são suficientes para que se possa descrever as informações de aplicações reais, principalmente quando se trata de estruturas mais complexas.

Nestes casos, pode ser definido os próprios tipos do usuário. A declaração de um tipo novo é feita pela diretiva **typedef**, seguida do tipo base e do novo nome.

```
typedef <tipo> novoTipo;
```

Onde:

<tipo>	É o tipo base a ser utilizado na definição.
novoTipo	É o nome dado ao novo tipo.

A linguagem C não possui a definição para o tipo **byte**. Esse pode ser definido como um **unsigned char**. Ou definir que o tipo **dinheiro** é baseado no tipo float.

```
1 typedef unsigned char byte;
2 typedef float dinheiro;
```

Estas definições ajudam a organizar o código e facilitam a portabilidade. O valor float, por exemplo, pode possuir 4 bytes em uma arquitetura, mas em outra valer 8. Arrumando apenas a definição da variável dinheiro, é garantido que em todos os lugares em que esse tipo for utilizado, as variáveis sejam alteradas corretamente.

Uma das grandes vantagens do **typedef** é criar novos tipos em conjunto com as diretivas **enum**, **struct** ou **union**.

Exemplo da criação do tipo bool:

```
1 //definição do tipo bool
2 typedef enum {false, true} bool;
3
4 //criação de uma variável do tipo bool
5 bool condition;
```

Para o **bool**, a palavra **false** passa a valer zero por estar na primeira posição. Já a palavra **true** passa a ter valor um. No entanto, as variáveis criadas com o tipo **bool** não possuem apenas 1 bit de tamanho. Elas possuem, em geral, o mesmo tamanho de um inteiro, que dependendo da plataforma pode ser 8, 16, 32 ou 64 bits.

Outro tipo comum de tipo para ser definido é a porta. Por porta entendemos um modo de acesso do microcontrolador ao mundo externo. Em geral, as portas representam através de bits os terminais físicos. Nestas situações, escrever o valor 1 (um) em um bit faz com que um determinado dispositivo ligue. Escrever o valor 0 (zero), por sua vez, faz com que ele desligue. O trecho do código 5.1 apresenta essa definição.

Código 5.1: Criação de uma struct para acesso individual a cada bit de uma porta

```
1 typedef struct {
2     unsigned bit0:1 ;
3     unsigned bit1:1 ;
4     unsigned bit2:1 ;
5     unsigned bit3:1 ;
6     unsigned bit4:1 ;
7     unsigned bit5:1 ;
8     unsigned bit6:1 ;
9     unsigned bit7:1 ;
10 }port;
11
12 //criação de uma variável do tipo struct port
13 port A;
14 //liga dispositivo da porta A, bit 0
15 A.bit0 = 1;
16 //desliga dispositivo da porta A, bit 0
17 A.bit0 = 0;
```

Exercícios

5.6

Ex. 5.1 — Qual a diferença entre um vetor de inteiros com 3 posições e uma estrutura com 3 membros inteiros?

```
1 int vetor[3];
2 struct triplo{
3     int v0;
4     int v1;
5     int v2;
6 };
```

Ex. 5.2 — Defina o tipo **talher**. Ele é descrito por um enumerador com 3 valores: colher, garfo e faca.

Ex. 5.3 — Crie uma estrutura composta por três membros de 2 bits e dois membros de 2 bits.

Ex. 5.4 — Construa uma matriz de tamanho 10 x 10 do tipo **pontos**. Os pontos são definidos por uma estrutura com dois inteiros: **x** e **y**.

ESTRUTURAS CONDICIONAIS

- 7.1 COMANDO CONDICIONAL IF **104**
- 7.2 COMANDO CONDICIONAL ANINHADO **105**
- 7.3 COMANDO DE SELEÇÃO SWITCH...CASE **106**
- 7.4 EXERCÍCIOS **109**

“Tentar não. Faça, ou não faça. Tentativa não há.”

Mestre Yoda

No desenvolvimento de um programa é comum existirem algumas ações que devem ser executadas apenas quando alguma condição for verdadeira. Nestas situações são utilizadas estruturas de decisão.

As estruturas condicionais permitem que o programa escolha entre dois blocos de comando distintos para serem executados.

O bloco de comando é um conjunto de comandos que devem ser executados juntos. Um bloco de comandos pode ser utilizado em qualquer trecho de programa que se pode usar um comando C. O bloco é delimitado por chaves, tendo seu início marcado por um “{” e é finalizado por um “}”.

7.1 Comando condicional if

A estrutura condicional mais simples é a **if**. O comando **if** realiza a decisão de mudança do fluxo do código baseada em uma expressão. Se a expressão for verdadeira os comandos serão executados. Se a expressão for falsa os comandos são ignorados.

A expressão condicional pode ser formada por qualquer tipo de código ou operação cujo resultado seja um valor. Se este valor for igual a zero, a expressão é definida como falsa. Para qualquer outro valor, diferente de zero, a expressão é avaliada como sendo verdadeira.

A sintaxe do **if** pode ser descrita como:

```
if(exp){  
    cmd;  
} //end if
```

Onde:

exp	Representa a condição a ser avaliada como verdadeira ou falsa.
cmd	Lista de comandos a serem executados caso a expressão seja verdadeira.

A estrutura **if** também pode ser acompanhada de um segundo bloco de comandos que será executado apenas quando a expressão for falsa. Este segundo bloco é iniciado pela palavra reservada **else**, conforme a sintaxe:

```
if(exp){  
    cmd_1; //bloco verdade  
}else{  
    cmd_2; //bloco falso  
}
```

Onde:

exp	Representa a condição a ser avaliada.
cmd_1	Comandos a serem executados caso exp seja verdadeira.
cmd_2	Comandos a serem executados caso exp seja falsa.

A expressão dentro dos parênteses do **if** é avaliada e se for verdadeira (valor de expressão diferente de zero), o primeiro bloco de comandos, o bloco verdade, é executado. Caso a expressão seja falsa (valor igual a zero), o segundo bloco de comandos, o bloco falso, é executado.

O trecho do código a seguir apresenta um exemplo de uso da estrutura de decisão.

```
1 if(temperature > 20){
2     turnOnCooler();
3     turnOffHeater();
4 }else{
5     turnOnHeater();
6     turnOffCooler();
7 }
```

A expressão avaliada é (**temperature > 20**). Esta expressão é verdadeira sempre que o valor armazenado na variável **temperature** for maior que 20. Neste caso, o cooler será ligado e o aquecedor desligado, comandos pertencentes ao primeiro bloco de comandos. Se a expressão avaliada for falsa, ou seja, se a variável **temperature** for menor ou igual a 20, o segundo bloco será executado, desligando o cooler e ligando o aquecedor.

Apenas o código associado ao **if** ou o código associado ao **else** será executado, nunca ambos.

7.2 Comando condicional aninhado

Em alguns casos, apenas uma expressão não é suficiente para implementar o que se deseja que o sistema faça. Nestas situações, pode-se encadear mais de uma operação if.

```
1 if(expressao1){
2     //bloco verdade do if 1
3     lista de comandos;
4
5     if(expressao2){
6         //expr.1 verdadeira e expr.2 verdade
7         lista de comandos;
8     }else{
9         //expr.1 verdadeira e expr.2 falsa
10        lista de comandos;
11    }
12 }else{
13     //bloco falso do if 1
14     lista de comandos;
15
16     if(expressao3){
17         //expr.1 falsa e expr.3 verdadeira
18         lista de comandos;
19     } else{
20         //expr.1 falsa e expr.3 falsa
21         lista de comandos;
22     }
23 }
```

A sintaxe de um **if** aninhado é muito similar aos **if**'s normais. A diferença é que agora se podem tomar decisões que dependem do resultado de uma outra decisão.

Na sintaxe apresentada, existe um **if** aninhado em cada bloco de comandos do primeiro **if**. Isto não é obrigatório. Pode-se colocar um **if** apenas no bloco verdade ou apenas no bloco falso. Isto fica a critério do programador.

Um exemplo seria um sistema de controle de temperatura. Além de ligar e desligar os sistemas de cooler e aquecimento, pode ter condições que, devido a temperaturas muito altas ou muito baixas, exigem algum tipo de alarme. Se for necessário ajustar a temperatura em 20 graus, com alarmes para temperaturas maiores que 30 ou menores que 10 graus, pode-se codificar esse comportamento de acordo com o trecho do código 7.1.

Código 7.1: Exemplo de uso de if aninhado para controle de temperatura

```
1 if(temperature > 20){
2     turnOnCooler();
3     turnOffHeater();
4     if(temperature > 30){
5         setAlarmHigh();
6     }
7 }else{
8     turnOffCooler();
9     turnOnHeater();
10    if(temperature < 10){
11        setAlarmLow();
12    }
13 }
```

Comando de seleção *switch...case*

7.3

Uma situação bastante comum é a necessidade de se comparar uma mesma variável com um grande conjunto de valores, de modo que cada valor execute uma ação diferente. Um exemplo deste comportamento é o processo de leitura dos botões de um sistema. Para cada botão pressionado uma ação diferente pode ser executada: o botão “start” deve executar a ação selecionada, o botão “pra_esquerda” deve voltar a tela anterior, o botão “A” deve aumentar o valor da variável selecionada.

Para realizar essas diversas comparações com uma mesma variável pode ser usada a estrutura **switch...case**. A comparação só pode ser feita com números e caracteres, sendo apenas comparações de igualdade, não sendo possível testar se uma variável é maior ou menor que um valor. Cada comparação **case** também é conhecida como um caso.

A sintaxe de utilização da estrutura **switch...case** é dada por:

```
switch( variavel ){
    case 1:
        //se variavel == 1
        lista de comandos 1;
        break;

    case 2:
        //se variavel == 2
        lista de comandos 2;
        break;

    //adiciona-se quantos cases forem necessários
    case 57:
        //se variavel == 57
        lista de comandos 57;
        break;

    default:
        //se for diferente de todos os valores acima
        lista de comandos default;
        break;
} //end switch
```

Onde:

variavel	É a variável a ser testada.
case_n	Executa uma lista de comandos caso a variável possua o valor <i>n</i> .
default	Se a variável não possuir nenhum dos valores anteriores a lista de comandos default será executada.

Na estrutura **switch...case** pode ser criada quantas condições forem necessárias. No entanto, essas condições servem apenas para comparações com números ou letras, não sendo permitida a comparação da variável em teste com uma segunda variável.

Todas as estruturas **case** devem possuir um **break**. A ausência do **break** acionará um efeito conhecido como fallthrough. Nesta situação, o código continuará a ser executado até encontrar o próximo **break**.

Existe também um caso especial que é definido com a palavra **default**. Em português, **default** pode ser traduzido como padrão. Este é o caso executado quando nenhuma comparação for verdadeira.

Como boa prática de programação deve-se utilizar sempre um caso padrão **default**.

Um bom exemplo de uso desta estrutura é a execução de comandos que devem ser executados apenas sob uma dada entrada no teclado. Supondo o desenvolvimento de um jogo, onde poderia ser definido as funções dos botões do joystick como apresentado no trecho do código 7.2.

Apesar de neste exemplo não ser executada nenhuma ação quando não há teclas pressionadas,

Código 7.2: Exemplo de uso da estrutura case para leitura de um teclado

```
1 //mapeando as teclas
2 //A - pulo, B - corrida, X - tiro, Y - tiro forte
3 //S'tart - pausa/resume
4 key = kbReadKey();
5 switch( key ){
6     case 'A':
7         jump();
8         break;
9     case 'B':
10        dash();
11        break;
12     case 'X':
13        shoot();
14        break;
15     case 'Y':
16        if(powerShootCount > 0){
17            powerShoot();
18            powerShootCount--;
19        }
20        break;
21     case 'S':
22        if(pauseStatus == 1){
23            unpause();
24            pauseStatus = 0;
25        }else{
26            pause();
27            pauseStatus = 1;
28        }
29        break;
30     default:
31        //nenhum comando acionado
32        break;
33 }//end switch
```

ainda assim é uma boa prática inserir o caso **default**.

Para dois casos, “Y” e “S”, é inserido uma estrutura **if** dentro do **case**. No “Y” esta estrutura verifica primeiro se o jogador não gastou todos os tiros especiais antes de executar a ação **powerShoot()**.

O botão “S” start pode realizar duas operações dependendo do estado atual do jogo: pausar ou continuar. Para isso, é utilizada uma variável auxiliar, **pauseStatus**, que indica o estado atual do jogo, servindo de comparação e controle para este botão.

7.4 Exercícios

Ex. 7.1 — Dados três pontos quaisquer: $A(x_A, y_A)$, $B(x_B, y_B)$, $C(x_C, y_C)$, eles serão colineares se pertencerem a uma mesma reta. Para determinar se os 3 pontos são colineares, pode-se verificar se os coeficientes angulares das retas formadas pelos pontos AB e pelos pontos BC são iguais, ou seja, $y_A - y_B / x_A - x_B = y_B - y_C / x_B - x_C$. Construa um programa usando a linguagem C que, dados 3 pontos quaisquer, seja capaz de responder se são ou não colineares.

Ex. 7.2 — Um professor deseja automatizar seu sistema de pontuação. Ele aplica 5 (cinco) provas e calcula o conceito do aluno usando apenas as quatro maiores notas. Uma média de 9.0 ou mais ganha um grau A; 8.0 a 8.9, um grau B; 7.0 a 7.9, um grau C; e 0.0 a 6.9, um grau D. Escreva um programa usando a linguagem C que, dadas as 5 notas, encontre as quatro maiores, calcule a média e descubra o conceito do aluno.

Ex. 7.3 — Faça um programa usando a linguagem C para descobrir o menor valor entre três valores numéricos.

Ex. 7.4 — Uma equação quadrática é da forma $ax^2 + bx + c = 0$, onde $a \neq 0$. As duas soluções para esta equação são dadas por $x = \frac{-b \pm \sqrt{b^2 - 4ac}}{2a}$: onde a quantidade $(b^2 - 4ac)$ é denominada discriminante da equação. Escreva um programa usando a linguagem C que, dados os respectivos coeficientes (a, b e c), calcule o discriminante e imprima as soluções. Use as seguintes regras: discriminante = 0 raízes iguais; discriminante < 0 não há solução real; discriminante > 0 duas soluções reais distintas.

Ex. 7.5 — Faça um programa usando a linguagem C, que, dado um número, calcule se este número é par ou ímpar e se é positivo ou negativo.

Ex. 7.6 — Faça um programa usando a linguagem C, que, dado o ano de nascimento de uma pessoa, diga se ela poderá ou não votar este ano. No Brasil, a pessoa com mais de 16 anos pode votar. Não é necessário considerar o mês em que a pessoa nasceu.

Ex. 7.7 — As maçãs custam R\$1,30 cada, se forem compradas menos de uma dúzia, e R\$1,00, se forem compradas pelo menos 12. Escreva um programa usando a linguagem C, que dado o número de maçãs compradas, calcule e escreva o custo total da compra.

Ex. 7.8 — Um dado sensor de corrente, que monitora a corrente de um motor, foi conectado ao microcontrolador através de um conversor analógico-digital. A corrente é medida em miliamperes e pode ser lida através do endereço `0x5f4`. Caso o motor trave, ou seja bloqueado por algum motivo, a corrente sobe demasiadamente. Para evitar a queima do sistema, o motor deve ser desligado nesta situação. Faça um programa cíclico que monitore a corrente do sistema. Quando essa corrente ultrapassar o valor de 512 miliamperes o sistema deve desligar o bit 2 da porta A, que, por sua vez, desliga o motor. A porta A pode ser acessada pela variável `PORTA`.

ESTRUTURAS DE REPETIÇÃO

- 8.1 REPETIÇÃO COM TESTE NO INÍCIO 112
- 8.2 REPETIÇÃO COM TESTE NO FINAL 112
- 8.3 REPETIÇÃO COM VARIÁVEL DE CONTROLE 113
- 8.4 COMANDOS DE DESVIO 115
- 8.5 ROTINAS DE TEMPO 115
- 8.6 EXERCÍCIOS 117

“Para atingir a excelência em qualquer coisa na vida, é preciso repetir e treinar. Treinar e repetir, aprender a técnica de tal maneira que ela se torne intuitiva.”

Paulo Coelho, *Aleph*

Existem estruturas que permitem a repetição de um trecho de código várias vezes. Estas estruturas são bastante utilizadas quando uma determinada sequência de ações ou eventos tem que acontecer um determinado número de vezes, ou enquanto uma determinada condição for verdadeira. Em computação é comum chamar estas estruturas de repetição ou **loops**.

Todo loop **deve** possuir uma condição que indique quando ele deve terminar. Uma condição mal feita, que nunca é falsa, pode fazer com que o programa fique preso indeterminadamente. Esta é uma das causas mais comuns para o “travamento” das aplicações, chamada também de **loop infinito**.

8.1 Repetição com teste no início

A repetição com teste no início do loop é usada quando se deseja que um determinado conjunto de comandos seja repetido enquanto uma condição for verdadeira. A sintaxe esta estrutura em linguagem C é dada por:

```
while( condicao ){  
    comandos;  
    alteracaoDaCondicao;  
}//end while
```

Onde:

condição	Indica ao processador se o loop deve continuar a ser executado.
comandos	Lista de comandos a serem executados caso a condição seja verdadeira.
alteraçãoDaCondição	Comando para alterar o valor da condição.

O controle do loop é feito através de uma condição. Para que o sistema não entre em "loop infinito" esta condição tem que ser alterada em algum momento dentro do loop.

Pode ser utilizada esta estrutura para ficar aguardando o jogador pressionar algum botão conforme apresentado no trecho do código 8.1.

Código 8.1: Exemplo de uso da estrutura while para leitura de um teclado

```
1 int x = kpRead();  
2 //0 indica que não houve botão pressionado  
3 while(x == 0){  
4     x = kpRead();  
5 }
```

Repetição com teste no final

8.2

Assim como a instrução **while**, a instrução **do...while** é utilizada para repetir um bloco do algoritmo diversas vezes. A diferença entre as duas é onde a verificação da condição é realizada. Na primeira, o teste é feito antes de entrar no loop. A segunda executa o loop uma vez e depois faz o teste.

Mesmo que a condição seja falsa desde o começo da execução do programa, na estrutura **do...while** o bloco é executado pelo menos uma vez.

A sintaxe desta estrutura é bastante similar à do **while**, como pode ser visto no trecho do código a seguir:

<pre>do{ comandos; alteracao da condicao }while(condicao);</pre>	
Onde:	condição Indica ao processador se o loop deve continuar a ser executado.
	comandos Lista de comandos a serem executados caso a condição seja verdadeira.
	alteraçãoDaCondição Comando para alterar o valor da condição.

O controle do loop também é feito através de uma condição. Portanto, é necessário que essa condição seja alterada dentro do loop.

Como esta estrutura faz o teste apenas ao fim do loop, executando o código dentro do loop pelo menos uma vez, ela é uma alternativa mais simples para escrever o mesmo código de leitura do joystick, como pode ser visto no trecho do código 8.2.

Código 8.2: Exemplo de uso da estrutura do...while para leitura de um teclado

```
1 //não precisa executar a leitura antes como no while  
2 do{  
3     x = kpRead();  
4     //zero indica que não houve botão pressionado  
5 }while(x == 0);
```

8.3 Repetição com variável de controle

Em vários casos é preciso executar um conjunto de comandos uma certa quantidade de vezes. Uma maneira de se obter esse resultado é utilizar uma variável de controle em conjunto com as estruturas de repetição já apresentadas. A variável de controle será responsável por indicar quantas vezes o loop já foi realizado. O trecho do código 8.3 apresenta como implementar esse comportamento utilizando a estrutura **while**.

No trecho do código 8.3, a variável **i** é iniciada com o valor zero antes do loop começar. A condição de execução do loop é dada por **i<10**, que é verdadeira, já que o valor de **i** é zero.

Dentro do loop são executados os comandos necessários, **doStuff()** e a variável **i** é incrementada em uma unidade. Quando o loop voltar à instrução **while**, a condição continuará verdadeira, **1<10**, e o loop será executado novamente. Este processo se repetirá mais nove vezes.

Na décima rodada a variável **i** terá o valor nove. Como nove é menor que dez, o loop é executado mais uma vez. Ao fim do loop o valor da variável será aumentado para dez. Na próxima

execução do loop o resultado será falso, o que fará com que o loop seja encerrado e o programa continue.

Código 8.3: Repetição de um código dez vezes com a estrutura while

```
1 int i;
2
3 i=0; // começa a contagem
4 while(i<10){
5     doStuff();
6     i++;
7 }
```

Esta é uma estrutura bastante comum em diversas atividades, principalmente em operações com vetores. Por isso foi criada uma estrutura que simplifica a execução de loops controlados por uma variável. Esta estrutura é denominada **for** e sua sintaxe é dada por:

```
for(inicialização; condição; alteração){
    comandos;
}
```

Onde:

inicialização	É um conjunto de comandos a serem realizados antes de começar o loop.
condição	Indica quando o loop deve parar de executar.
alteração	É um conjunto de comandos a serem executados entre cada iteração do loop. Serve normalmente para alterar o valor da condição.
comandos	Lista de comandos a serem executados dentro do loop.

Esta estrutura é uma forma de resumir o exemplo apresentado anteriormente com a estrutura **while** no trecho do código 8.3. Reescrevendo esse código com a estrutura **for**, tem-se o trecho do código 8.4.

Código 8.4: Repetição de um código dez vezes com a estrutura for

```
1 int i;
2 for(i=0; i<10; i++){
3     doStuff();
4 }
```

Ambos os códigos são executados praticamente com a mesma velocidade e consumindo a mesma quantidade de memória. A vantagem da estrutura **for** é apresentar de maneira mais simples os valores de início, fim e alteração da variável de controle.

Pode-se utilizar a estrutura **for** para imprimir cada posição de um vetor de letras no display de LCD. Para isso, é definido uma mensagem e é utilizada a função **LcdChar()**, que imprime um caractere de cada vez. O trecho do código 8.5 apresenta a implementação desta rotina.

Código 8.5: Repetição de um código dez vezes com a estrutura while

```
1 //o texto "Hello World" possui 11 caracteres
2
3 //a variável message possui 12 posições para armazenar também o caracter ↵
  de terminação '\0'
4 char message[] = "Hello World";
5
6 //como não é necessário imprimir o '\0', paramos a contagem no décimo ↵
  primeiro elemento
7
8 for (i=0; i<11; i++) {
9     lcdChar(message[i]);
10 }
```

8.4 Comandos de desvio

Existem dois comandos que podem modificar o andamento dos loops. Eles podem ser utilizados dentro do bloco de comandos de qualquer tipo de laço. São eles:

- **break** - provoca a saída do laço em que se encontra no momento.
- **continue** - termina a execução da lista de comandos e reinicia o loop. Se as condições continuarem válidas, uma nova iteração é realizada.

Os comandos **break** e **continue** permitem ao programador alterar o fluxo do programa dentro de um loop.

Ao utilizar o comando **break**, o loop é parado imediatamente, independentemente das condições. O programa tem sequência no primeiro comando depois do loop. Também é utilizado em conjunto com o comando **switch**.

Ao utilizar o comando **continue**, a iteração atual do loop para de ser executada no ponto onde o comando foi chamado e o loop recomeça. Se for usada dentro de um loop do tipo **for**, o bloco de incremento é executado antes de recomeçar o loop.

Rotinas de tempo

8.5

Trabalhar com requisitos temporais é muito comum em sistemas embarcados. Algumas atividades devem ser executadas com uma certa cadência, com um tempo entre elas. Se estes tempos não forem obedecidos, não é possível fazer com que o sistema funcione corretamente.

Apesar de existirem periféricos dedicados à contagem de tempo, é muito comum utilizar uma estrutura de repetição para isso. A estrutura apenas fará com que o processador conte a partir de zero até um valor pré-definido, como no código a seguir:

```
1 unsigned char i;
2 for(i=0; i < 100; i++);
```

O código apresentado faz apenas a contagem, para a variável `i`, de zero a noventa e nove, totalizando cem repetições. Dependendo da velocidade do processador, do tipo da variável e dos valores programados, o loop pode demorar mais ou menos tempo para ser finalizado. Para as três plataformas, por exemplo, temos os tempos deste loop reunidos na tabela 8.1.

Tabela 8.1: Tempo gasto para execução de 100 loops por tipo de variável, em microssegundos

Plataforma	<code>char</code>	<code>volatile char</code>	<code>int</code>	<code>volatile int</code>	<code>float</code>	<code>volatile float</code>
Arduino Uno r3	29,0	70,1	42,2	120,8	1137,0	1268,1
Chipkit Uno32	0,8	14,4	0,8	12,1	108,7	170,3
Freedom KL05	74,1	83,0	70,4	70,6	887,2	887,5

Se for preciso fazer contagens de tempo maiores pode-se utilizar estruturas de repetição encaeadas. Deste modo, os valores são multiplicados. A plataforma Arduino, por exemplo, consome 29,0 uS para um loop com 100 repetições. Para gerar um atraso aproximado de 1 segundo pode-se executar este loop 34.483 vezes ($34483 \times 29\mu S = 1,000007s$). Para conseguir contar até esse valor é necessário uma variável do tipo `unsigned int`.

```
1 void delay1s(void){
2     char i;
3     unsigned int j;
4     for(j=0; j<34483; j++) { //34483*29 = 1,000007s
5         for(i=0; i<100; i++); //100 repetições = 29uS
6     }
7 }
```

Executando o código acima na plataforma Arduino observa-se um tempo de 17,37mS, muito diferente do tempo de 1 segundo calculado. Percebendo que o loop não executa nenhuma tarefa, o compilador otimiza o processo, omitindo alguns cálculos. Deste modo, o tempo consumido fica menor que o esperado.

Para evitar esse problema, podem ser utilizadas as variáveis com o modificador `volatile`. Assim, a contagem de tempo será bem mais próxima do esperado. Para a plataforma Arduino, como a contagem com uma variável do tipo `volatile char` consome 70,1 uS, deve ser feito um loop externo de 14.266 para atingir 1 segundo.

```
1 void delay1s(void){
2     volatile char i;
3     volatile unsigned int j;
4     for(j=0; j<14266; j++) { //14266*70,1 = 1,00005s
5         for(i=0; i<100; i++); //100 repetições = 70,1uS
6     }
7 }
```

No teste real, o loop consumiu 1,011 segundos, um erro de apenas 1% do valor 1,00005 calculado.

lado anteriormente. As variáveis com o modificador **volatile** não são otimizadas durante o processo de compilação, possuindo um comportamento mais previsível.

Como pode ser visto, acertar os valores de tempo de modo exato, pode consumir muito tempo de desenvolvimento. Esta abordagem não é uma boa solução para contagem precisa de tempo. No entanto, é uma alternativa simples quando se precisa gerar um atraso, sem compromisso com a precisão, ou quando os hardwares dedicados (timers e RTC's) não estiverem disponíveis.

Para contagens mais simples, que não exijam uma precisão muito alta, pode-se criar uma função que permita gerar um atraso programável. Para isto, basta executar uma estrutura de repetição e ajustar os valores até obtermos uma base de tempo razoavelmente confiável. Depois disso, estrutura-se a função para repetir o código de acordo com a quantidade requisitada.

```
1 void delay1ms(void){
2     char i;
3     unsigned int j;
4     for(j=0; j<15; j++) {          //15*70,1 ~ 1,05us
5         for(i=0; i<100; i++);      //100 repetições = 70,1uS
6     }
7 }
8 void delay(unsigned int time){
9     while(time--){ //executa time vezes.
10        delay1ms();
11    }
12 }
```

8.6 Exercícios

Ex. 8.1 — Construa um programa usando a linguagem C que calcula a diferença entre a diagonal principal (células do canto superior esquerdo ao canto inferior direito) com a diagonal secundária (células do canto superior direito ao canto inferior esquerdo) de uma matriz de 5000 linhas por 5000 colunas.

Ex. 8.2 — Uma pista de atletismo circular é formada por uma quantidade de raias concêntricas (mesmo centro), consecutivas e com a mesma largura. A partir do valor do raio que marca o início da primeira raia, pode-se calcular o valor do raio que marca o início da segunda raia adicionando o valor da largura e assim sucessivamente. Construa um programa usando a linguagem C que possibilita calcular a distância que será percorrida para completar uma volta em cada marca de raia na pista definida pelo usuário. São dados: a fórmula para cálculo do perímetro da circunferência ($p = 2 \times \pi \times r$) e o valor do raio.

Ex. 8.3 — Sendo a matriz A de ordem “m x n” e a matriz B de ordem “n x r”, o produto da matriz A por B será uma matriz C de ordem “m x r” cujos elementos são obtidos pela soma dos produtos das linhas de A pelas colunas de B. Assim, um elemento na matriz C é calculado por $c[i][j] = a[i][1].b[1][j] + a[i][2].b[2][j] + \dots + a[i][n].b[n][j]$.

Escreva um programa usando a linguagem C que calcula a multiplicação da matriz A de ordem 100 x 30 e da matriz B de ordem 30 x 50, armazenando o resultado em uma matriz C.

Ex. 8.4 — Uma faculdade deseja saber quais são os alunos que cursam a disciplina Cálculo, mas não estão cursando a disciplina de Matemática Discreta. Existe disponível um vetor que armazena as matrículas dos alunos que fazem Cálculo (com 50 alunos) e um outro vetor que armazena as matrículas dos alunos de Matemática Discreta (com 80 alunos).

Ex. 8.5 — Em estatística, o conceito de variância é usado para descrever um conjunto de observações. Quando o conjunto de observações é uma população, é chamada de variância da população. Se o conjunto das observações é (apenas) uma amostra estatística, é chamado de variância amostral (ou variância da amostra). Faça um programa usando a linguagem C que, dado um vetor V com 20 elementos, calcule a variância da amostra (Var) usando a seguinte fórmula: $Var(X) = \sum_{i=1}^n p_i \cdot (v_i - \mu)^2$, onde: v_i é o elemento i do vetor V; n é o número de elementos; e μ é a média de todos os elementos, que deve ser calculada antes.

Ex. 8.6 — Apresentar o total da soma dos cem primeiros números pares ($2 + 4 + 6 + \dots + 98 + 100$).

Ex. 8.7 — Faça um programa usando a linguagem C que calcula o número de divisores inteiros de um dado número.

Ex. 8.8 — Um número primo é qualquer inteiro positivo divisível apenas por si mesmo e por 1. Escreva um programa usando a linguagem C que calcula se um dado número é primo ou não.

Ex. 8.9 — Dado um número natural n , faça um programa usando a linguagem C para determinar e escrever o número harmônico H_n definido por: $H_n = 1 + 1/2 + 1/3 + 1/4 + \dots + 1/n$.

Ex. 8.10 — Faça um programa usando a linguagem C que dado um valor N inteiro e positivo, calcule o fatorial de N ($N!$).

Ex. 8.11 — Faça um programa usando a linguagem C que calcule o resultado da expressão de potenciação, dado o valor de uma base e da potência.

Ex. 8.12 — Faça um programa usando a linguagem C que calcule todos os números múltiplos de 7 entre 1 e N , sendo N um valor definido no começo do programa. Por exemplo: 7, 14, 21, 28, 35.

Ex. 8.13 — Construa um programa usando a linguagem C que, dados dois vetores X e Y, encontre entre estes 1000 pontos quais são os 3 pontos cuja soma entre suas distâncias seja o valor máximo. A fórmula para calcular a distância entre dois pontos é: $dist = \sqrt{((x_a^2 - x_b^2) + (y_a^2 - y_b^2))}$. A soma entre 3 pontos A, B e C é dada por $dist(A,B) + dist(B,C) + dist(C,A)$. Essa soma deve ser a maior entre os 1000 pontos disponíveis nos vetores.

Ex. 8.14 — Dois apostadores A e B apostaram cartelas na mega-sena. Cada cartela na mega-sena contém 6 números distintos. O apostador A está concorrendo com 150 cartelas e o apostador B está concorrendo com 95 cartelas. Para facilitar, suponha que as cartelas já estão preenchidas, não existem números repetidos na mesma cartela nem cartelas repetidas para o mesmo jogador. No entanto, os valores dos números de uma cartela não obedece nenhuma ordem. O programa deve descobrir quantas das cartelas do apostador A são iguais à do apostador B. Devem-se utilizar as matrizes a e b definidas abaixo para armazenar as cartelas do apostador A e do apostador B.

```
1  int a[150][6];
```

Ex. 8.15 — Faça um programa usando a linguagem C que verifica se um dado número faz parte da sequência de Fibonacci. Os dois primeiros números na sequência de Fibonacci são 1 e 1. O n -ésimo número é dado pela seguinte fórmula: $F_n = F_{n-1} + F_{n-2}$, para $n > 2$. Por exemplo, os oito primeiros números da sequência são 1, 1, 2, 3, 5, 8, 13 e 21. Assim, o número 7 não faz parte da sequência de Fibonacci, bem como o número 13 faz parte.

Ex. 8.16 — A diretoria de uma faculdade deseja saber se existem alunos cursando as disciplinas de Algoritmos e Linguagens de Programação, mas que não estão cursando Estrutura de Dados. Existe disponível um vetor que armazena as matrículas dos alunos que fazem Algoritmos (50 alunos); um outro vetor que armazena as matrículas dos alunos de Linguagens de Programação (80 alunos); e um outro vetor que armazena as matrículas dos alunos de Estrutura de Dados (80 alunos). Faça um programa usando a linguagem C que realiza a busca utilizando os três vetores dados.

```
1 int algo[50];
2 int prog[80];
3 int ed[80];
```

Ex. 8.17 — Dado um conjunto de 100 valores numéricos, faça um programa usando a linguagem C que calcula o somatório dos 100 valores dados obedecendo a fórmula a seguir:

$$S = (b1 + b100)^2 + (b2 + b99)^2 + \dots + (b50 + b51)^2$$

Ex. 8.18 — Faça um programa usando a linguagem C que armazena em um vetor de dimensão 60, os números que representam as notas dos alunos. Depois descubra qual é a média e a maior nota.

Ex. 8.19 — Faça um programa usando a linguagem C para corrigir 30 provas de múltipla escolha. Cada prova tem 10 questões, cada questão valendo um ponto. O primeiro conjunto de dados será o gabarito para a correção da prova. Os outros dados serão respostas dos 30 alunos.

Ex. 8.20 — Faça um programa usando a linguagem C que inicializa duas matrizes de inteiros A e B, cada uma de duas dimensões com 5 linhas e 3 colunas. Construa uma matriz C de mesma dimensão, onde C é formada pela soma dos elementos da matriz A com os elementos da matriz B.

Ex. 8.21 — Faça um programa usando a linguagem C que inicializa uma matriz identidade de ordem 1000. A matriz identidade é uma matriz onde todos os elementos são zero, com exceção dos elementos da diagonal principal que valem 1.

Ex. 8.22 — Crie um programa que leia um valor de distância de um sensor ultrassônico na entrada analógica e apresente a distância no barramento de LEDs (porta D). Este sensor consegue identificar distâncias de 40 cm à 200 cm. Quando há um objeto a 50 cm, a saída do sensor apresenta 0 volts (0 no AD). Quando o objeto está à distância de 200 cm, a saída fica saturada em 5 volts (1024 no AD). O sensor possui uma resposta linear. O programa deve deixar todos os leds acesos quando o sensor indicar objeto a 50 cm e apagar um led a cada 20 cm, de modo

que todos os leds fiquem apagados quando o objeto estiver a 200 cm ou mais. Pode-se fazer uso da biblioteca `adc.h`.

```
1 //adc.h
2 void InicializaADC(void);
3 unsigned int LeValorAD(void);
```

Ex. 8.23 — Um modelo simples de cifragem é a utilização de uma operação XOR dos dados a serem cifrados com um número. Para realizar a decifragem dos dados basta fazer novamente a operação XOR com o mesmo número. Este número pode ser entendido como uma senha. Este sistema apresenta um nível de segurança muito ruim. No entanto, é bastante rápido, sendo útil em algumas situações. Supondo que a região onde os dados estão gravados na memória vai de `0x100` até `0x2FE`, faça um programa que cifre estes dados, realizando a operação de XOR com o número `0xEF`, e armazene os dados cifrados na região de memória de `0x300` até `0x4FE`.

```
1 //exemplo de cifragem
2 dadoCifrado = dadoNãoCifrado ^ 0xEF;
```

Ex. 8.24 — Existe uma variável não sinalizada, com 16 bits (**unsigned int**) e com o nome “TECLADO”. Cada bit desta variável representa uma tecla física. Se a tecla está pressionada seu valor é **1**. Se estiver solta o valor do bit é **0**. Construa um programa cíclico que verifique quais teclas estão pressionadas e exiba no LCD. Para isso, pode ser utilizada a função **lcdNumber(int n)**.

Ex. 8.25 — Em um dado sistema, a biblioteca do GPS é composta pelos arquivos “gps.h” e “gps.c” e a biblioteca de controle do quadricóptero é “quad.h” e “quad.c”. Na biblioteca GPS existem apenas três funções: “void InicializaGPS(void)”, que inicializa todas as configurações necessárias, “int LerLatitude(void)”, que retorna o valor da latitude, e “int LerLongitude(void)”. Caso não haja caractere, a função retorna o valor 0 (zero). Na biblioteca quad também têm três funções: “void VelNorteSul(int vel)”, que altera a velocidade do quadricóptero na direção norte, se o valor for positivo, e para a direção sul, se for negativo. A função “void VelLesteOeste(int vel)”, recebe um parâmetro indicando em qual velocidade o quad se deslocará na direção leste (positivo) e oeste (negativo). Por último, a função “void InicializaQuad(void)”, responsável por configurar o quadricóptero. Supondo que o lugar que o quadricóptero será manobrado não possui nenhum obstáculo, construa um programa cíclico que verifique a posição atual do quad e controle-o para que chegue ao seu destino. A posição destino já está definida em duas variáveis: “int longDest;” e “int latDest;”.

CIRCUITOS UTILIZADOS NAS EXPERIÊNCIAS

O aprendizado em programação embarcada envolve o contato tanto com o software quanto com o hardware. As placas de controle utilizadas ao longo do livro (Arduino, Chipkit e Freedom) facilitam a utilização da parte eletrônica, entregando uma solução simples de gravação/regração do firmware do microcontrolador. Elas também provêm um caminho de comunicação serial e acesso aos periféricos internos.

Para os periféricos externos é necessário montar os circuitos adjacentes ao microcontrolador. O esquemático destes circuitos foi apresentado ao longo dos capítulos do livro. Para simplificar as atividades práticas estes circuitos podem ser montados tanto em protoboards ou utilizar a placa de desenvolvimento projetada para o livro.

São ao todo seis circuitos externos: o led RGB, o display quádruplo de 7 segmentos, o teclado matricial 2x5, o display de LCD, os sensores analógicos com o buzzer e o RTC DS1307.

As conexões com o led RGB são bastante simples. Elas utilizam os mesmos terminais de ativação dos displays de 7 segmentos. Os resistores utilizados são de 1k, conforme figura 1:

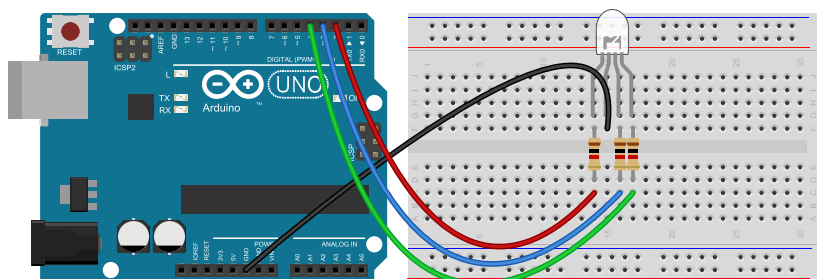


Figura 1: Ligação em protoboard do circuito do LED RGB

Fonte: Imagem produzida com Fritzing/Inkscape

O circuito de conexão dos displays de 7 segmentos exige o uso do 74HC595. Três linhas de comando são enviadas a ele, que, por sua vez, transforma os dados seriais das três linhas em 8 saídas digitais, que seguem para os displays. O acionamento de cada display é feito por quatro transistores controlados individualmente pelo microcontrolador. A conexão no protoboard segue conforme a figura 2:

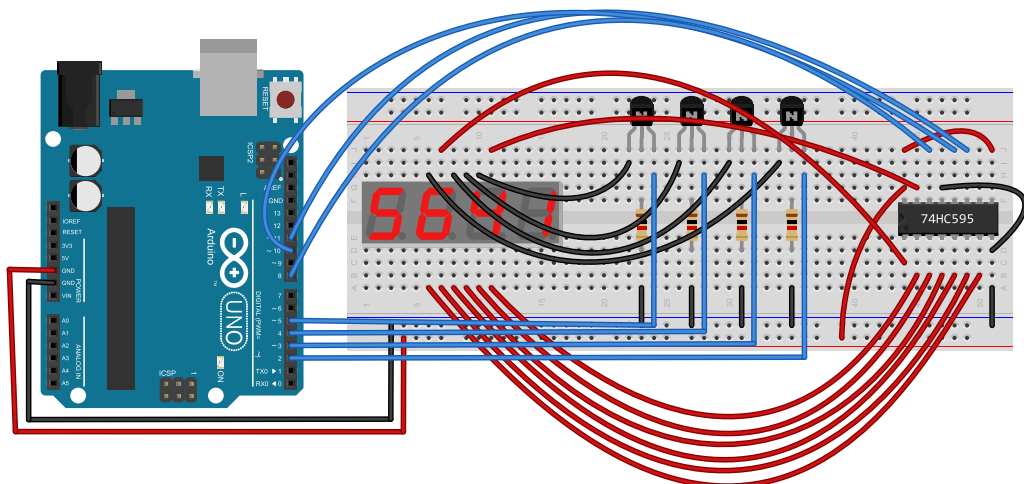


Figura 2: Ligação em protoboard do circuito de display de 7 segmentos
 Fonte: Imagem produzida com Fritzing/Inkscape

O circuito de acionamento do LCD é bastante parecido com o do 7 segmentos. A diferença é que os sinais do 74HC595 seguem para o display de LCD. O LCD também exige um potenciômetro para controle de contraste, que é conectado ao terminal de número 3. Todos os terminais não utilizados são conectados ao terra. A conexão no protoboard segue conforme a figura 3:

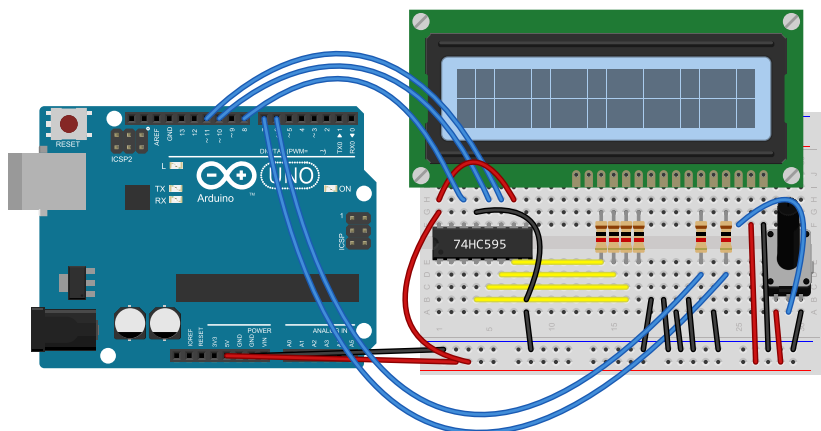


Figura 3: Ligação em protoboard do circuito do display de LCD
 Fonte: Imagem produzida com Fritzing/Inkscape

A conexão do teclado exige alguns cuidados extras. As teclas possuem curto interno entre dois de seus quatro terminais, por isso as teclas que estão conectadas no centro do protoboard possuem seus contatos na vertical e as conectadas em cima ou embaixo têm seus contatos na horizontal. Os dados novamente saem do 74HC595, mas seguem para um conjunto de diodos, que servem tanto para proteção quanto para evitar acionamentos indevidos das teclas. Os sinais saem dos diodos e, passando pelas chaves, voltam a se reunir nos transistores, que farão a conversão de tensão para a leitura das teclas pelo microcontrolador. A conexão no protoboard segue conforme a figura 4:

Os dispositivos analógicos são bastante simples de serem conectados. Em geral, é necessário

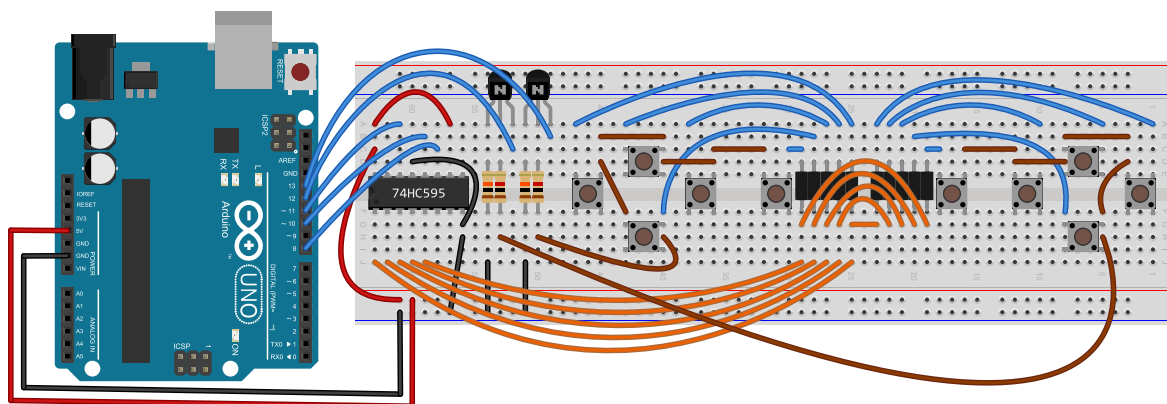


Figura 4: Ligação em protoboard do circuito de leitura do teclado matricial 5x2
 Fonte: Imagem produzida com Fritzing/Inkscape

levar apenas a alimentação (GND e VCC) a eles e conectar a saída no terminal de leitura analógica da placa. O buzzer é utilizado com um circuito transistorizado para seu acionamento. A conexão no protoboard segue conforme a figura 5:

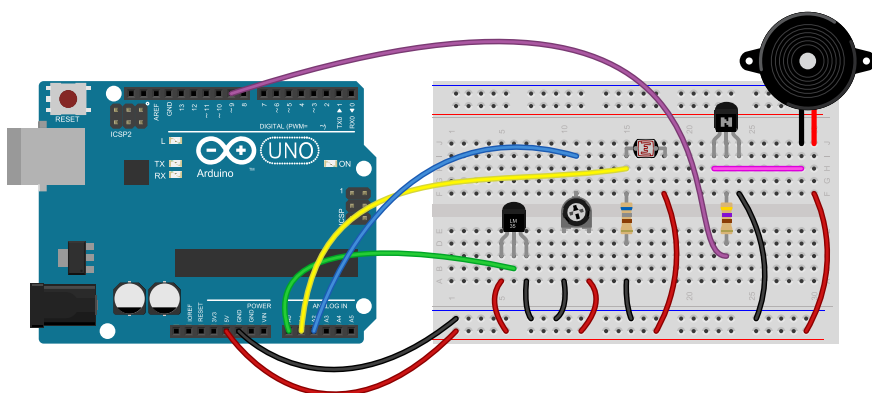


Figura 5: Ligação em protoboard dos circuitos sensores analógicos e do buzzer
 Fonte: Imagem produzida com Fritzing/Inkscape

O último circuito externo utilizado é o relógio de tempo real DS1307. Ele se comunica via protocolo I²C com a placa de controle, por isso os dois terminais de dados possuem os resistores de pull up. Além da alimentação da placa, o circuito conta com a possibilidade de adição de uma bateria externa, evitando perder a contagem de tempo quando a placa for desligada. Por fim um cristal dedicado é adicionado para o correto funcionamento do relógio. A conexão no protoboard segue conforme a figura 6.

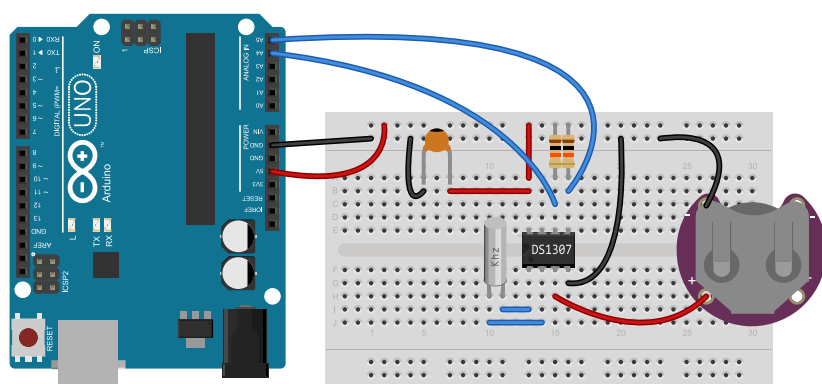


Figura 6: Ligação em protoboard do circuito do relógio DS1307
Fonte: Imagem produzida com Fritzing/Inkscape



PROJETO DA PLACA DE DESENVOLVIMENTO

A placa de desenvolvimento foi projetada para conter todos os dispositivos necessários para apresentar os conceitos abordados nesse livro. Por limitações de terminais disponíveis no barramento das placas compatíveis com o barramento do Arduino, foi utilizado um circuito de expansão de terminais que faz a conversão de serial para paralelo.

Outra preocupação foi deixar a placa base compatível com circuitos de 5 ou 3,3 volts. Esta providência foi tomada para poder abranger o maior número de placas e permitir o uso da placa base sem restrições.

No entanto, essa abordagem pode levantar alguns problemas: queimar a placa base ou a placa de processamento, se as tensões não foram compatíveis.

Para evitar esse problema, a placa base não deverá sofrer nenhuma avaria se for utilizado uma placa de processamento com tensões de 5 volts e a placa de desenvolvimento não poderá causar nenhum dano às placas de processamento de 3,3 volts. As soluções encontradas foram:

1. Alimentar todos os componentes externos da placa de desenvolvimento em 5 volts. A maioria dos componentes externos possui capacidade de operar em ambas as tensões fornecidas pela placa (5v e 3,3v). No entanto, alimentar os componentes em 3,3v e enviar sinais de controle em 5 volts poderia causar danos. O contrário, alimentar em 5v e enviar sinais de 3,3v não causa problemas físicos (queima).
2. Utilizar circuitos digitais que, mesmo alimentados em 5 volts, aceitem a tensão de 3,3 volts como nível alto. Deste modo se a tensão do sinal for zero volts, ele será considerado de nível baixo. Se a tensão for tanto 3,3 quanto 5 volts será considerado de nível alto.
3. Os sinais digitais que sairão da placa de desenvolvimento serão todos em 3,3 volts, assim as placas de controle não serão afetadas. Do mesmo modo que os componentes, as placas de controle entendem o nível de 3,3v como alto, mesmo as alimentadas em 5 volts. Os sinais digitais utilizarão um transistor para efetuar a mudança dos níveis de tensão para 3,3 volts.
4. Os sinais analógicos devem possuir valores que poderão variar entre zero e 3,3v. Quando for utilizada uma placa de processamento de 5 volts, será perdido um pouco da faixa de conversão, não utilizando toda a escala do conversor. Isto significa uma perda de resolução do sinal, o que, para aplicações didáticas, não compromete o uso.

O esquemático apresentado na figura 7 foi desenvolvido levando todos os requisitos em conta.

Para simplificar o projeto, depuração, fabricação e montagem da placa, a numeração dos componentes segue um padrão. Todos eles são identificados por um conjunto de letras mais três dígitos numéricos. As letras indicam o tipo de componente. O primeiro dígito representa de qual circuito o componente faz parte. Os dois últimos dígitos são números sequenciais para diferenciar os componentes dentro do mesmo circuito. Os circuitos foram divididos em:

- Entradas digitais;
- Entradas analógicas e saída PWM;
- Display de 7 segmentos;
- Display de LCD;
- Led RGB;
- Conversor serial-paralelo (74HC595);
- Circuito do RTC DS1307;
- Shield com footprint do Arduino.

Com relação às letras utilizadas na identificação dos componentes temos:

- R - Resistências;
- C - Capacitores;
- U - Circuitos integrados / chips;
- D - Diodos;
- X - Cristal de quartzo;
- SHIELD - Terminais de conexão;
- LDR - Sensor de luminosidade;
- RV - Potenciômetro;
- Q - Transistores;
- RBG - Led com três cores;
- BZ - Buzzer;
- BT - Bateria;
- SW - Chaves do tipo microswitch.

Com relação ao projeto do layout da placa, este foi planejado para ser possível de ser fabricado em face única. Isto permite a confecção caseira da placa de modo rápido e simples. Além disso, todos os traços possuem a espessura mínima de 15mils (0,381mm), com isolamento de 30mils (0,762mm). Isto evita reduzir a possibilidade de curtos ou problemas de corrosão na fabricação caseira da placa. Por esse motivo algumas providências tiveram que ser tomadas para garantir essas premissas:

- O LCD faz a comunicação através de um barramento de 4 bits. Isto reduz a quantidade de trilhas necessárias e facilita o roteamento.
- Os leds do display de 7 segmentos estão conectados de acordo com o posicionamento dos terminais físicos. Por isso a sequência dos segmentos na saída do 74HC595 é dada por **ed0cga**fb****, e não por **0g**fedc**ba**.
- O footprint (desenhos do componente) utilizado para os diodos foi alongado de modo que houvesse mais espaço entre os seus terminais. Deste modo, criou-se um caminho livre embaixo dos diodos que permitiu a passagem de sinais do lado esquerdo para o lado direito da placa.

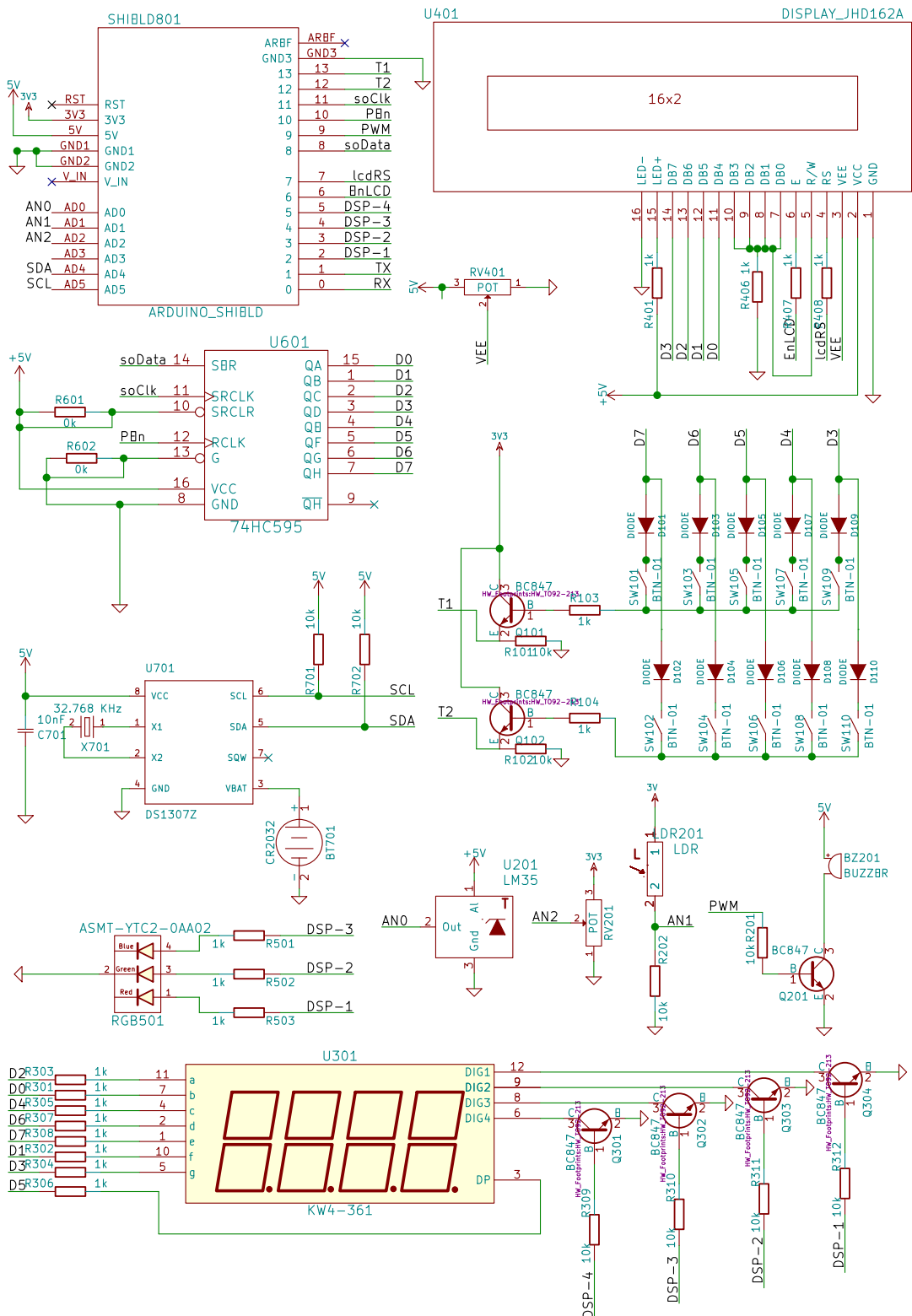


Figura 7: Esquemático completo da placa de desenvolvimento

- Dois resistores de zero ohms foram adicionados (R601 e R602). Eles funcionam como jumpers para conectar as trilhas em regiões onde não foi possível contornar os caminhos dos sinais.
- O espaço entre os terminais dos componentes não foi utilizado para a passagem de trilhas visando simplificar a fabricação da placa e respeitar os espaçamentos definidos.

O posicionamento dos componentes é apresentado na figura 8 (fora de escala). O layout final do traçado é apresentado, em escala, na figura 9

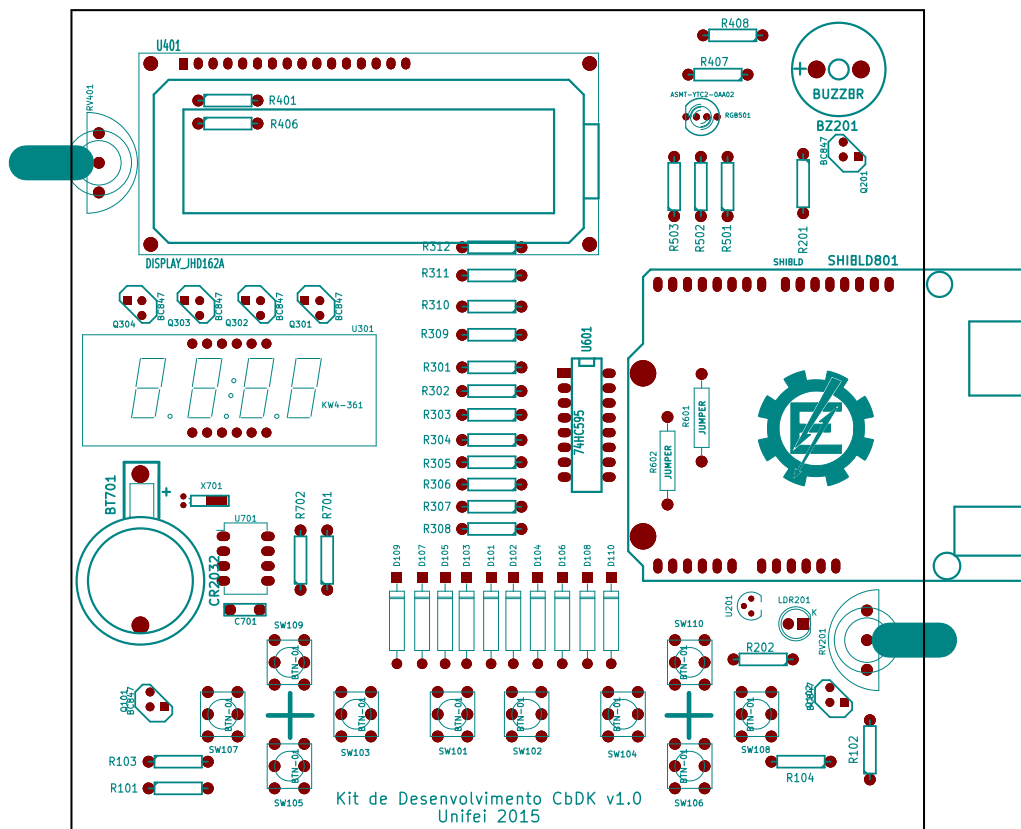


Figura 8: Posicionamento dos componentes

A lista de componentes utilizados, contendo sua identificação e sua descrição, é apresentada na tabela 1. Os resistores R601 e R602 possuem a resistência de zero ohms pois são apenas fios. Eles foram utilizados para permitir que a placa fosse produzida em face simples.

Ao todo, para montar a placa são necessários:

- 15 Resistores 1k
- 10 Resistores 10k
- 2 Resistores (jumpers) 0 ohm
- 1 capacitor 10nF
- 1 cristal 32,768kHz
- 1 74HC595

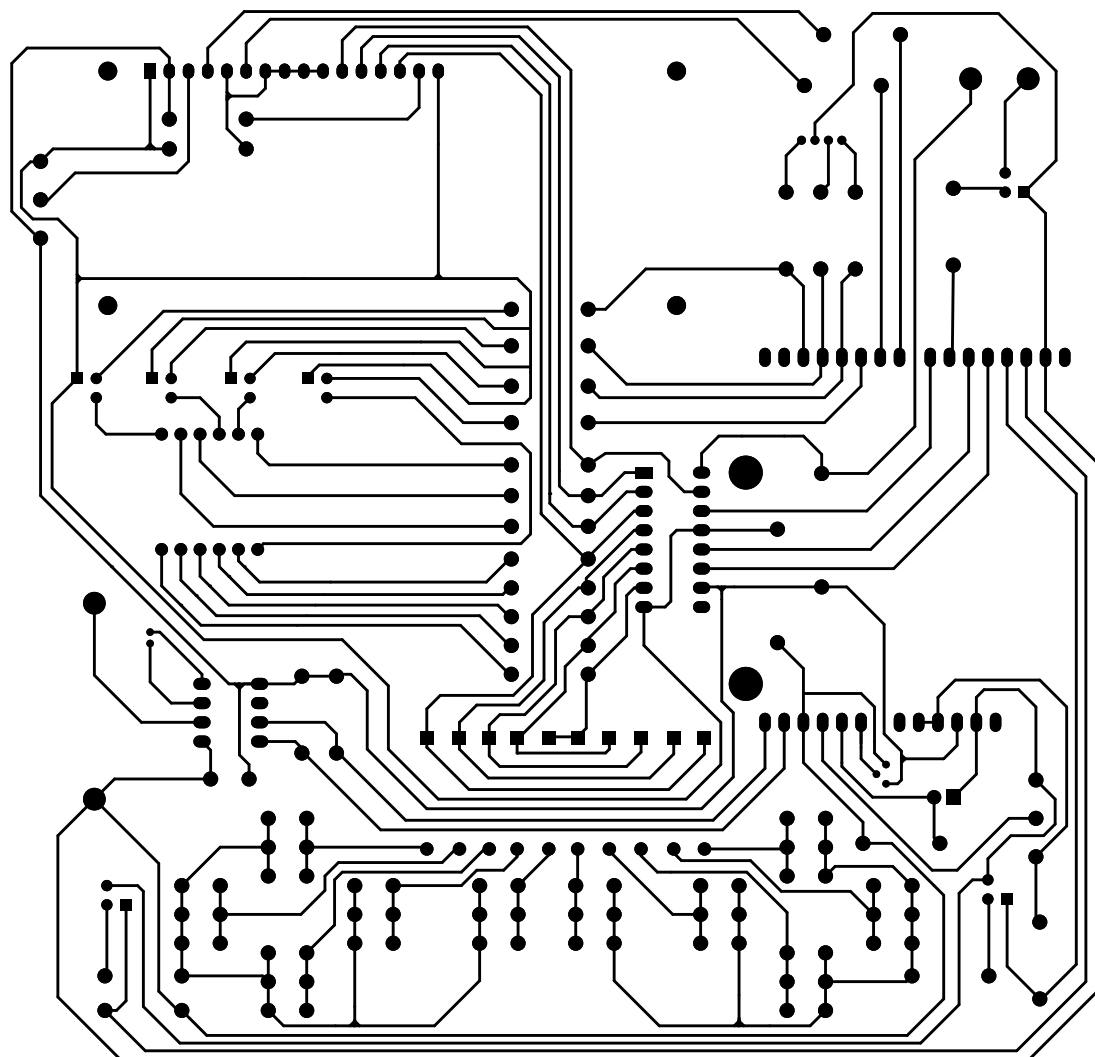


Figura 9: Layout da placa de desenvolvimento

- 1 placa de controle Arduino/Chipkit/-Freedom
- 28 terminais macho, divididos em 4 barras: 6, 6, 8 e 8 pinos cada.
- 1 Led RGB
- 7 transistores BC337
- 10 chaves microswitch, 2 ou 4 terminais
- 1 buzzer
- 10 diodos, 1N4148 ou similar
- 1 display LCD 12x6 JHD162A ou compatível
- 1 display quádruplo 7 segmentos, KW4-361 ou compatível
- 1 LDR 10k
- 2 potenciômetros 10k

O modelo 3D do circuito é apresentado na figura 10. O posicionamento dos botões foi modelado para simular um controle de vídeo-game. Os dispositivos de exibição (LCD e sete segmentos)

são posicionados na parte superior da placa. O shield se localiza lateralmente para permitir conexão dos cabos de alimentação e comunicação. Os potenciômetros estão na borda para facilitar o acesso. Os demais componentes foram colocados de modo a minimizar problemas com roteamento.

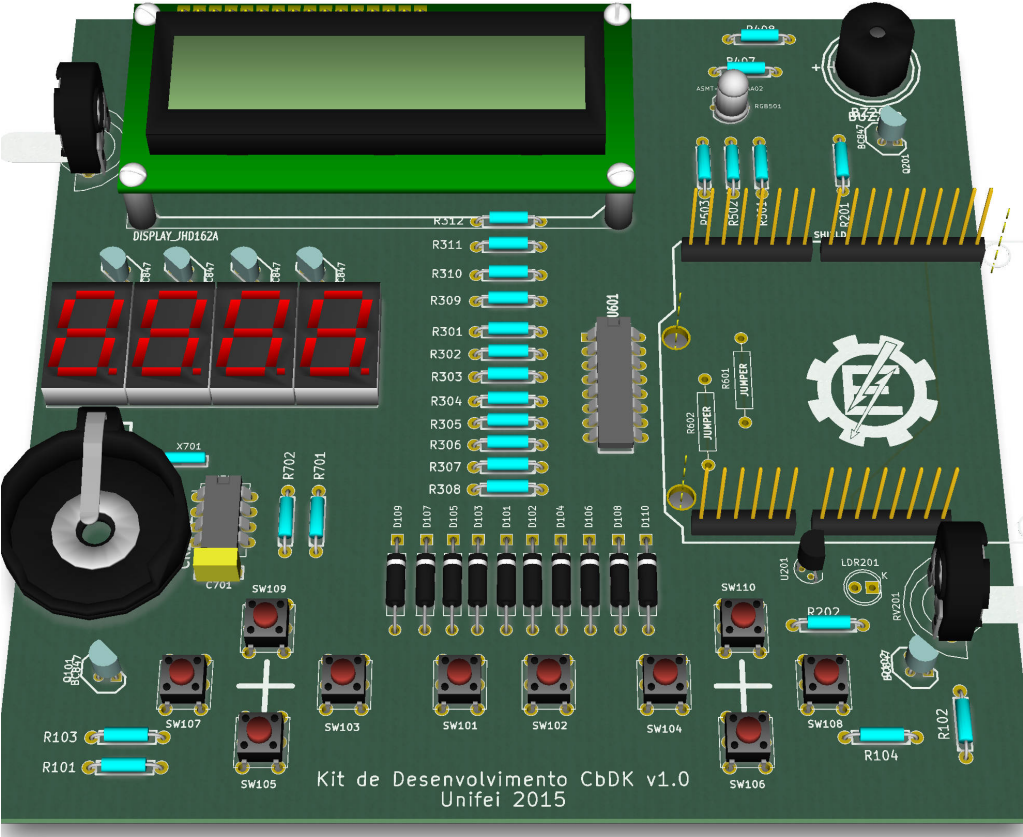


Figura 10: Renderização 3d da placa com os componentes adicionados.

Tabela 1: Componentes utilizados e descrição

ID	Item	ID	Item	ID	Item
BT701	CR2032	R101	10k	R502	1k
BZ201	BUZZER	R102	10k	R503	1k
C701	10nF	R103	1k	R601	0k
D101	1n4148	R104	1k	R602	0k
D102	1n4148	R201	10k	R701	10k
D103	1n4148	R202	10k	R702	10k
D104	1n4148	R301	1k	RGB501	Led RGB
D105	1n4148	R302	1k	SW101	Microswitch
D106	1n4148	R303	1k	SW102	Microswitch
D107	1n4148	R304	1k	SW103	Microswitch
D108	1n4148	R305	1k	SW104	Microswitch
D109	1n4148	R306	1k	SW105	Microswitch
D110	1n4148	R307	1k	SW106	Microswitch
LDR201	LDR	R308	1k	SW107	Microswitch
Q101	BC847	R309	10k	SW108	Microswitch
Q102	BC847	R310	10k	SW109	Microswitch
Q201	BC847	R311	10k	SW110	Microswitch
Q301	BC847	R312	10k	U201	LM35
Q302	BC847	R401	1k	U301	Display quádruplo 7 segmentos
Q303	BC847	R406	1k	U401	Display LCD16x2
Q304	BC847	R407	1k	U601	74HC595
RV201	Potenciôme- tro 10k	R408	1k	U701	DS1307Z
RV401	Potenciôme- tro 10k	R501	1k	X701	Cristal 32.768 kHz

EXERCÍCIOS RESOLVIDOS

Resolução do ex. 6.2 — Pode-se implementar um XOR lógico da seguinte maneira:

```
1 #define XOR(a,b) ((a && (!b)) || (b && (!a)) )
```

Resolução do ex. 6.3 — Nas operações Lógicas, os operandos são tomados como um todo, nas operações Bitwise, os operandos são processados bit a bit.

```
1 ec0 = 0;  
2 ec1 = 1;  
3 ec2 = 3; //ou 0b11  
4 ec3 = 1;  
5 ec4 = 0;  
6 ec5 = 1;  
7 ec6 = 255; //ou 0xFF  
8 ec7 = 1;  
9 ec8 = 0;  
10 ec9 = 240; //ou 0xF0  
11 eca = 0;  
12 ecb = 237; //ou 0b11101101  
13 ecc = 0;  
14 ecd = 90; //ou 0x5A  
15 ece = 1; //ou (0b11) & 0b01 = 0b01  
16 ecf = 1; //ou (0b00) | 0b01 = 0b01
```

Resolução do ex. 7.2 — Inicializam-se as variáveis com valores aleatórios apenas para testes.

```
1 void main (void){  
2     int n1,n2,n3,n4,n5;  
3     int media;  
4  
5     lcdInit();  
6     n1 = 5;
```

```

7     n2 = 7.4;
8     n3 = 1.3;
9     n4 = 9.8;
10    n5 = 7.2;
11    media = (n1+n2+n3+n4+5)/5;
12    if(media>9.0){
13        lcdChar('A');
14    }else if(media>8.0){
15        lcdChar('B');
16    }else if(media>7.0){
17        lcdChar('C');
18    }else
19        lcdChar('D');
20    }
21 }

```

Resolução do ex. 7.8 — Deve-se criar uma variável do tipo ponteiro para apontar para o endereço pedido pelo exercício.

```

1 void main (void){
2     //como o valor é maior que 256 a variável deve ser int
3     int *corrente;
4     //aponta para o endereço da variável corrente
5     corrente = 0x5f4;
6     for(;;){
7         if(*corrente > 512){
8             BitClr(PORTA,2);
9         }
10    }
11 }

```

Resolução do ex. 8.10 — Para calcular o valor do fatorial, basta multiplicar o número por todos os valores de 1 até N. Deve-se tomar o cuidado de não incluir o valor 0.

```

1 void main (void){
2     int N;
3     int fat;
4     int i;
5
6     //inicialização do sistema
7     systemInit();
8     lcdInit();
9
10    N = 10;
11    fat = 1;
12    //deve começar com 1 e não com zero para não zerar o resultado
13    for(i=1; i<=N; i++){
14        fat = fat * i;
15    }

```

```
16     lcdNumber(i);
17     for(;;);
18 }
```

Resolução do ex. 8.20 — O primeiro loop inicialize as matrizes **A** e **B** com valores baseados em **i** e **j**, já que o exercício não pediu nenhum valor específico. O segundo loop realiza o cálculo.

```
1 void main(void){
2     int i, j, matA[5][3], matB[5][3], matC[5][3];
3     //inicializacao das matrizes A e B
4     for(i=0; i<5; i++){
5         for(j=0; j<3; j++){
6             matA[i][j] = i+j;
7             matB[i][j] = i*j;
8         }//end for j
9     }//end for i
10    //calculo da matriz matC
11    for(i=0; i<5; i++){
12        for(j=0; j<3; j++){
13            matC[i][j] = matA[i][j] + matB[i][j];
14        }//end for j
15    }//end for i
16 }//end main
```

Resolução do ex. 8.21 — Antes de armazenar o valor da matriz, é verificado se o item atual é um item da diagonal principal. Isso acontece quando **i==j**. Neste caso, salva-se o valor **1**.

```
1 void main(void){
2     int i, j, matI[1000][1000];
3     //inicializacao da matriz identidade
4     for(i=0; i<1000; i++){
5         for(j=0; j<1000; j++){
6             if(i==j){
7                 matI[i][j] = 1;
8             }else{
9                 matI[i][j] = 0;
10            }//end else
11        }//end for j
12    }//end for i
13 }//end main
```

Resolução do ex. 8.23 — A variável **a** é um ponteiro que terá o valor de seu endereço apontado para onde os dados estão. Para salvar o valor cifrado, basta somar o valor **0x200** antes de derreferenciar o ponteiro: por exemplo, **0x200 -> 0x400**, **0x235 -> 0x435**.

```
1 void main (void){
2     unsigned char *a;
3     for(a=0x100; a<0x2ff; a++){
```

```

4      *(a+0x200)=*a ^ 0xFE;
5  }
6 }

```

Resolução do ex. 8.24 — Para simplificar o código, pode ser usado um loop para testar cada uma das posições através da função **bitTst**. Para cada tecla pressionada, envia-se o valor para o LCD.

```

1 void main (void){
2     int i;
3     for(;;){
4         for(i=0;i<16;i++){
5             if(bitTst(TECLADO,i){
6                 lcdNumber(i);
7             }//end if
8         }//end for i
9     }//end for
10 }

```

Resolução do ex. 8.25 — Existem diversos modos de realizar o controle. O mais econômico é criar uma linha reta entre o início e o fim. O mais simples é corrigir primeiro a longitude e depois a latitude. A resposta é baseada na segunda abordagem. Enquanto houver erro, o sistema corrige no sentido contrário: se a latitude atual é menor, move o quadcóptero para o norte, se for maior, para o sul.

```

1 #include "gps.h"
2 #include "quad.h"
3
4 //longitude -> esquerda/direita
5 //latitude-> norte/sul
6 Void main (void){
7     //inicializa com os valores do alvo
8     int longDest=10;
9     int latDest=-5;
10
11     InicializaQuad();
12     InicializaGPS();
13
14     //executa os comandos em loops para acertar primeiro a longitude e ←
15     //depois a latitude
16     for(;;){
17         //se o quad estiver para "baixo", deve ir pro norte chegar na ←
18         //mesma linha.
19         while(longDest > LerLongitude()){
20             VelNorteSul(+1);//anda até chegar na longitude correta
21         }//end while
22         //se o quad estiver para "cima", tem que ir para o sul
23         while(longDest < LerLongitude()){
24             VelNorteSul(-1);//anda até chegar na longitude correta
25         }//end while
26     }
27 }

```

```

23         }//end while
24         //se estiver a esquerda
25         while(latDest > LerLatitude()){
26             VelLesteOeste(+1);
27         }//end while
28         //se estiver a direita
29         while(latDest < LerLatitude()){
30             VelLesteOeste(-1);
31         }//end while
32     }//end for
33 }//end main

```

Resolução do ex. 9.4 — A criação do header deve ser feita apresentando todos os protótipos das funções. As variáveis que devem armazenar valores dentro da biblioteca são definidas com o modificador **static**, dentro do arquivo de código e fora de qualquer função, para serem acessadas por todas as funções do código.

```

1  //alarmes.h
2  #ifndef ALARMES_H
3  #define ALARMES_H
4      void valorTemp(int val);
5      void valorPres(int val);
6      void inicializaAlarme(void);
7      void atualizaAlarme(void);
8  #endif
9
10 //alarmes.c
11 #include "alarmes.h"
12 #include "basico.h"
13 static int alarmeTemp;
14 static int alarmePres;
15
16 void valorTemp(int val){
17     int alarmeTemp = val;
18 }//end valorTemp
19 void valorPres(int val){
20     int alarmePress = val;
21 }//end valorPres
22 void inicializaAlarme(void){
23     inicializaADC();
24 }//end inicializaAlarme
25 void atualizaAlarme(void){
26     if(LerValorAD(3) > alarmeTemp){
27         BitSet(PORTE,3);//liga alarme
28     }//end if
29     if(LerValorAD(6) > alarmePress){
30         BitSet(PORTE,6);//liga alarme
31     }//end if
32 }//end atualizaAlarme

```

Resolução do ex. 13.6 — As condições para ligar a prensa têm que acontecer simultaneamente. Se apenas uma delas estiver ligada, deve-se esperar que as duas sejam soltas para testar novamente.

```
1 #include "io.h"
2 void main(void){
3     TRISE = 0xDF; //apenas bit 5 é saída, 7 4 3 e 2 são entradas
4     for(;;){
5         //apenas bt 2 ativado, espera os dois serem soltos
6         if(bitTst(PORTE,2) && !(bitTst(PORTE,3)){
7             while(bitTst(PORTE,2) || (bitTst(PORTE,3)));
8         }//end if
9         //apenas bt 3 ativado, espera os dois serem soltos
10        if(!(bitTst(PORTE,2)) && bitTst(PORTE,3){
11            while(bitTst(PORTE,2) || (bitTst(PORTE,3)));
12        }//end if
13
14        //itens de segurança ativados
15        if(bitTst(PORTE,4) && bitTst(PORTE,7)){
16            //botões pressionados ao mesmo tempo
17            if(bitTst(PORTE,2) && bitTst(PORTE,3)){
18                //ativa prensa
19                bitSet(PORTE,5);
20                bitClr(PORTE,5);
21            }//end if
22        }//end if
23    }//end for
24 }//end main
```

Resolução do ex. 14.4 — Considerando entrada **1** e saída **0**, Binário: **0b00010110**, Hexadecimal: **0x16**, Decimal: **22**.

Resolução do ex. 14.6 — Além de inicializar adequadamente o conversor AD, o programa deve realizar a leitura e converter corretamente a escala. Neste caso, é necessário fazer uma conversão de **0-1023** para **0-8**. Depois deve-se ligar todos os leds e desligar apenas a quantidade correta, pois, quando a entrada é zero, todos os leds estão acesos (zero apagados) e, quando a entrada é **1023**, todos os oito estão apagados.

```
1 #include "adc.h"
2
3 void main(void){
4     char i;
5     int dist;
6     adInit();
7     TRISB = 0x00;
8     for(;;){
9         dist = adRead();
10        //transforma a escala de 0 à 1023 para 0 à 8
11        dist = (dist * 8) / 1023;
12        PORTB = 0x00; //acende os 8 leds
```



```

13     for(i=0; i<dist; i++){
14         //apaga uma quantidade de leds relativa a distância
15         BitClr (PORTB, i);
16     }//end for
17 }//end for
18 }//end main

```

Resolução do ex. 15.5 — Como os leds estão ligados em sequência direta com os terminais da porta (gfedbca), basta seguir apenas o fluxo normal ligando os leds da porta C de 0 a 6, já que o ponto decimal não conta.

```

1 void main(void){
2     char led;
3     int time;
4     TRISE = 0x2E; //habilitando como saída apenas os terminais dos ↵
        \textit{displays}
5     PORTE |= 0x2E; //ligando apenas os terminais dos \textit{displays}
6     TRISC = 0x00; //habilitando como saída os terminais dos leds
7
8     for(;;){        //\textit{loop} infinito
9         //contagem de tempo
10        for(time = 0; time<1000; time++);
11
12        //desligando todos os leds
13        PORTC = 0x00;
14        //liga apenas o led atual
15        BitSet(PORTC, led);
16        led++;
17        if(led >=7){
18            led = 0;
19        }
20    }
21 }

```

Resolução do ex. 16.4 — Como existem 3 linhas e três colunas, é necessário configurar os terminais adequadamente como saída ou entrada. O procedimento é o mesmo adotado na biblioteca **keypad**, com a diferença que o acionamento não é mais feito através do conversor serial-paralelo, mas diretamente nos terminais da porta.

```

1 //variável para armazenar o resultado
2 unsigned int Teclas;
3
4 void varreduraChaves(void){
5     unsigned char i,j;
6     TRISB = 0x07;           //configura pinos (210) como entrada
7     TRISC = 0x00;           //configura pinos (210) como saída
8     for(i=0; i<3; i++){
9         PORTC = 0x00;       //desliga todas as colunas
10        BitSet(PORTC,i);     //liga a coluna correspondente

```

```

11     for(j=0; j<3; j++){        //teste para cada bit, atualiza a variável
12         if(BitTst(PORTB,j)){
13             BitSet(valorNovo,(i*3)+j);
14         }else{
15             BitClr(valorNovo,(i*3)+j);
16         }//end else
17     }//end for
18 }//end for
19 }//end varreduraChaves

```

Resolução do ex. 16.5 — Suponha que os terminais são configurados com entrada em outra rotina de inicialização. A função *debounce* realiza a comparação com o valor inteiro da porta D, já que todos os 8 bits representam chaves.

```

1
2 char debounce(void){
3     char tecla = 0; //sem ninguém pressionado
4     char novatecla;
5     char tempo = 10;
6     char i;
7
8     //enquanto tempo não esgotar espera estabilizar
9     while(tempo){
10         novatecla = PORTD;
11         //testa se está estável
12         if(novatecla != tecla){
13             tecla = novatecla;
14             tempo = 10;
15         }else{
16             tempo--;
17         }//end else
18     }//end while
19
20     //deste ponto em diante tecla já está estável
21     for(i=0; i<8;i++){
22         //realiza o teste e retorna a primeira chave que encontrar ←
23         //pressionada
24         if(BitTst(tecla,i)){
25             return i+1;
26         }//end if
27     }//end for
28     return 0; //sem tecla pressionada
29 }//end debounce

```

Resolução do ex. 16.6 — Cada tecla pressionada incrementa o contador de valores. Por fim, um loop é utilizado para ligar a quantidade correta.

```

1 #include "so.h"
2 #include "io.h"

```

```

3 void main(void){
4     int n_leds, i;
5     unsigned char ledsAcesos;
6
7     //inicializa entradas
8     pinMode(D0,OUTPUT);
9     pinMode(D1,OUTPUT);
10    pinMode(D2,OUTPUT);
11
12    //inicializa conversor para poder acionar os leds
13    soInit();
14
15    for(;;){
16        n_leds = 0;
17        //teste led 1
18        if(digitalRead(D0)){
19            n_leds+=1;
20        }//end if
21        if(digitalRead(D1)){
22            n_leds+=2;
23        }//end if
24        if(digitalRead(D2)){
25            n_leds+=3;
26        }//end if
27        ledsAcesos = 0x00; //apaga todos os leds
28        //acende apenas a quantidade necessária
29        for(i=0; i<n_leds; i++){
30            BitSet(ledsAcesos,i);
31        }//end for
32        //envia o valor pelo conversor
33        soWrite(ledsAcesos);
34    }//end for
35 }//end main

```

Resolução do ex. 18.2 — Cada vez que a função **serialRead()** é chamada, o valor deve ser armazenado em uma variável temporária para realizar as comparações. Dependendo do valor a decisão correta é tomada.

```

1 #include "lcd.h"
2 #include "ssd.h"
3 #include "serial.h"
4
5 void main(void){
6     char txt;
7     int time;
8     adInit();
9     lcdInit();
10    ssdInit();
11    for(;;){
12        txt = serialRead();

```

```

13     if(txt>= '0' && txt <= '9'){
14         //converter de ASCII para número
15         ssdDigit(txt-48, 0);
16     }
17     if(txt>= 'A' && txt <= 'Z'){
18         lcdChar(txt, 0);
19     }
20     ssdUpdate();
21     for(time=0; time<1000; time++);
22 } //end for
23 } //end main

```

Resolução do ex. 18.4 — O primeiro passo é aguardar a chegada do número 42. Em qualquer outra situação o valor recebido é ignorado. A partir do momento que chegou o 42, qualquer valor recebido, com exceção de **0xff**, é adicionado a uma variável temporária. Quando chegar o número 24, finaliza, a soma e fica-se esperando o próximo valor diferente de **0xff**. Quando ele chegar, compara-se e verifica-se se a mensagem está correta.

```

1  #include "serial.h"
2
3  //suposições iniciais:
4  //não é necessário armazenar os valores da mensagem
5  //não é necessário tomar nenhuma ação quando a mensagem for válida ou ←
   inválida
6  //24 e 42 nunca são enviados como dados ou resultado de soma
7  //0xFF indica que não foi recebido valor pela serial
8
9  void main(void){
10     char msg_ok;
11     int temp, soma;
12
13     //inicializa os periféricos necessários
14     serialInit();
15
16     for(;;){
17         msg_ok = 0; //começa supondo mensagem errada
18         temp = serialRead();
19         //se recebeu começo da mensagem correta pode começar a ler e ←
   armazenar a soma
20         if(temp == 42){
21             soma = 0;
22             //enquanto temp for diferente de 24 (fim) continua a soma
23             while(temp != 24){
24                 temp = serialRead();
25                 //testa se chegou alguma coisa
26                 if(temp != 0xff){
27                     soma += temp;
28                 } //end if
29             } //end while
30

```

```

31         //quando chegar aqui o ultimo valor lido foi o 24.
32         //o próximo byte válido é o resultado da soma
33         do{
34             temp = serialRead();
35         }while(temp == 0xff);
36
37         //se a soma for igual a mensagem é valida
38         if(soma == temp){
39             msg_ok = 1;
40         }else{
41             msg_ok = 0;
42         }
43     }
44 } //end for
45 } //end main

```

Resolução do ex. 18.5 — A biblioteca console precisa receber uma string. Para isso, será criado um vetor de 2 posições onde a última sempre é um terminador `'\0'`. A primeira posição é preenchida com o valor recebido (se ele for diferente de 0xff).

```

1 void main(void){
2     //criando uma string para enviar para o console
3     char rx[2];
4     systemInit();
5     consoleInit();
6     serialInit();
7     //colocando o terminador na string
8     rx[1] = '\0';
9     for(;;){
10         //le o caracter da serial
11         rx[0] = serialRead();
12         //se for valido
13         if(rx[0] != 0xff){
14             //testa se é um dos caracteres de controle de movimento w/s
15             if(rx[0] == 'w'){
16                 consoleMoveLine(-1);
17             } else if(rx[0] == 's'){
18                 consoleMoveLine(+1);
19             }else{
20                 //se for valido mas não de movimentação, imprime no ←
21                 consolePrint(rx);
22             } //end else
23             consoleUpdate();
24         } //end if
25     } //end for
26 } //end main

```

Resolução do ex. 18.6 — O primeiro passo é conseguir criar um sistema que contabilize quantas

letras certas chegaram. Para isso, é utilizado um contador auxiliar. Para cada letra certa, o contador incrementa e uma nova condição **if** é habilitada. Se, em qualquer momento, chegar uma letra errada, o contador volta para zero.

```
1 #include "lcd.h"
2 #include "serial.h"
3
4 void main(void){
5     int qt_letras = 0; //conta quantas letras corretas já chegaram
6     char letra = 0;
7     serialInit();
8     lcdInit();
9     for(;;){ //loop infinito
10         //aguarda chegar uma letra
11         do{
12             letra = serialRead();
13         }while(letra == 0);
14         //testa se chegou 'n'
15         if(letra == 'I'){
16             qt_letra=1; //chegou uma letra certa
17         }
18         //testa se a letra atual é o 'D' e se já chegou 1 certa antes
19         if((letra == 'D')&& (qt_letra == 1)){
20             qt_letra=2; //já chegaram duas letras certas
21         }else{
22             qt_letras = 0; //chegou uma errada, tem que reiniciar
23         }
24         //testa se a letra é 'E' e se já chegou 2 certas antes
25         if((letra == 'E')&& (qt_letra == 2)){
26             qt_letra=3; //já chegaram três letras certa
27         }else{
28             qt_letras = 0; //chegou uma errada tem que reiniciar
29         }
30
31         //se chegou a palavra "IDE" via serial envia a identificação via ↵
32         serial
33         if(qt_letras == 3){
34             lcdString("11952");
35             qt_letras = 0;
36         }
37 }
```

Resolução do ex. 19.4 — Para armazenar os dados ao longo do tempo, foi criado um vetor de 5 posições. Eles iniciam com valor zero. O primeiro passo do loop é deslocar cada valor uma casa para trás. O último valor será perdido e a primeira posição ficará disponível para receber a nova medida. O valor filtrado é exibido no LCD.

```
1 #include "io.h"
2 #include "ad.h"
```

```

3 #include "lcd.h"
4
5 void main(void){
6     int adValue[5]; //armazena as cinco amostras
7     int i;
8     systemInit();
9     lcdInit();
10    adInit();
11    for(i=0; i<5; i++){
12        adValue[i] = 0; //inicializa todas as medidas com zero
13    }//end for
14    for(;;){
15        for(i=5; i>=1; i--){
16            adValue[i] = adValue[i-1]; //empurra todos os valores pra trás
17        }//end for
18        //leitura do potenciômetro
19        adValue[0] = analogRead(AN0);
20
21        //muda pra sempre imprimir na mesma posição
22        lcdPosition(0,0);
23        lcdNumber((adValue[0] + adValue[1] +
24                    adValue[2] + adValue[3] +
25                    adValue[4]) /5);
26    }//end for
27 }//end main

```

Resolução do ex. 20.4 — Para transmitir os dados adequadamente, o PWM tem que ser ligado durante 1 ou 2ms com a frequência desejada. Para cada bit será realizado um loop dentro do segundo **for**. Em todos os casos, a saída ficará desligada 1ms e ligada 1ms. Se o bit valer 1 ela fica ligada mais 1ms, totalizando os 2ms exigidos.

```

1 #include "io.h"
2 #include "pwm.h"
3
4 void main(void){
5     unsigned char data; //valor a ser transmitido
6     int i;
7
8     systemInit();
9     pwmInit();
10    pwmFrequency(38000);
11
12    for(;;){
13        for(i=0; i<8; i++){
14            //desliga o PWM
15            pwmDuty(0);
16            //gera o atraso de 1ms. Tem que ser ajustado de acordo com a ←
             plataforma
17            for(i=0; i<1000; i++);
18

```

```

19         //liga com duty cycle de 50%
20         pwmDuty(50);
21         //gera o atraso de 1ms. Tem que ser ajustado de acordo com a ←
           plataforma
22         for(i=0; i<1000; i++);
23
24         if(bitTst(data,i)){
25             //se for um bit 1 gasta mais 1ms, totalizando 2
26             for(i=0; i<1000; i++);
27         } //end if
28     } //end for
29 } //end for
30 } //end main

```

Resolução do ex. 22.1 — As interrupções são chamadas através de eventos. Esses eventos são geralmente iniciados por dispositivos de hardware. Deste modo, para definir quando a interrupção é chamada, devemos entender qual é o evento que dispara essa chamada. Para uma interrupção de timer, por exemplo, a função será chamada de modo periódico. Para uma interrupção que monitore uma entrada externa, ela será chamada quando houver mudança no valor do terminal.

Resolução do ex. 22.3 — Para a arquitetura apresentada no exercício, basta criar a função com o nome **SerialISR()** que automaticamente ela responderá às interrupções. Para receber o valor do dado, basta fazer a leitura no registro RX_DATA. O valor é armazenado em uma variável temporária para a comparação.

```

1 //variavel global
2 static unsigned char valor_serial;
3 void SerialISR(void){
4     unsigned char temp;
5     temp = RX_DATA;
6     if(temp >= '0' && temp <= '9'){
7         valor_serial = temp;
8     }else{
9         valor_serial = 0xFF;
10    } //end else
11 } //end SerialISR

```

Resolução do ex. 22.4 — Como a variável tick é global e efetua a contagem de tempo corretamente em milissegundos, basta realizar as conversões para segundos, minutos e horas e imprimir corretamente no lcd.

```

1 #include "io.h"
2
3 unsigned long int tick;
4 //início do programa
5 void main(void){
6     unsigned int seg;
7     unsigned int min;

```



```

8   unsigned int hora;
9   systemInit();
10  lcdInit();
11  for(;;){
12      //separando os segundos
13      seg = (tick/1000)%60;
14
15      //separando os minutos
16      min = (tick/1000/60)%60;
17
18      //separando as horas
19      hora = (tick/1000/60/60)%24;
20      //imprime as informações
21      lcdPosition(0,1);
22      lcdChar(hora/10+'0');
23      lcdChar(hora%10+'0');
24      lcdChar(':');
25      lcdChar(min/10+'0');
26      lcdChar(min%10+'0');
27      lcdChar(':');
28      lcdChar(seg/10+'0');
29      lcdChar(seg%10+'0');
30  } //end for
31 } //end main

```

Resolução do ex. 24.4 — Em um sistema operacional de tempo real, existem mecanismos que permitem ao programador garantir que a taxa de execução dos processos, ou de pelo menos parte dos processos, será cumprida.

Resolução do ex. 25.1 —

```

1  int qtd[POOLSIZE];
2  void main (void){
3      int i;
4      //zera as quantidades iniciais
5      for(i=0;i<POOLSIZE;i++){
6          qtd[i] = 0;
7      }
8
9      for(;;){
10         for(i=0;i<6;i++){
11             //chama a função e passa a quantidade de
12             //vezes que ela já foi executada como parâmetro
13             pool[i](qtd[i]);
14             qtd[i]++;
15         }
16     }
17 }

```

Resolução do ex. 25.2 —

```
1 char procLed(int n){
2     BitSet(PORTA, n);
3     if(BitTst(PORTA,n){
4         return SUCESS;
5     }else{
6         return FAIL;
7     }
8 }
```

Resolução do ex. 25.3 —

```
1 void main(void){
2     KernelInit();
3     KernelAddProc(LeSerial);
4     KernelAddProc(ProcessaBuffer);
5     KernelLoop();
6 }
```

Resolução do ex. 26.4 — O código é responsável por decrementar um temporizador interno de reexecução de cada um dos processos.

```
1 void KernelTick(void){
2     unsigned char i;
3     for(i=ini; i!=fim; i=(i+1)%SLOT_SIZE){
4         //apenas quando exige RT é necessário fazer contagem
5         if((pool[i].start)>(MIN_INT) && (ExigeRT))
6             pool[i].start--;
7     }
8 }
```

Resolução do ex. 27.3 — Como as funções básicas do driver devolvem informação, elas devem retornar o valor através do parâmetro, bastando realizar o teste nos bits adequados do teclado. Para a inicialização, deve-se configurar os terminais como entrada e ajustar as funções internas do driver. O restante do driver apenas segue o padrão definido.

```
1 //teclado.c
2
3 void LerTecla1(void *parameters){
4    >(* char)(parameters) = BitTst(PORTB,2);
5 }
6
7 void LerTecla2(void *parameters){
8    >(* char)(parameters) = BitTst(PORTA,4);
9 }
10
11 void InicializaTeclado(void *parameters){
```

```

12     BitSet(DDRA,4); //Set = Output
13     BitSet(DDRB,2);
14     thisDriver.drv_id = *(char*) parameters;
15     this_functions[TECLADO_TECLA1] = LerTecla1;
16     this_functions[TECLADO_TECLA2] = LerTecla2;
17 }
18
19 driver* getTecladoDriver(void){
20     //assegura que no mínimo a função init será conhecida
21     thisDriver.drv_init = InicializaTeclado;
22     thisDriver.func_ptr = (ptrFuncDrv*) & this_functions;
23     return &thisDriver;
24 }
25
26 //teclado.h
27
28 #ifndef TECLADO_H
29 #define TECLADO_H
30 #include "ddCtr_types.h"
31
32 enum{TECLADO_TECLA1, TECLADO_TECLA2, TECLADO_END};
33
34 //apenas retorna o "objeto" do driver
35 driver* getTecladoDriver(void);
36 #endif

```

Resolução do ex. 27.5 — Como as funções básicas do driver estão implementadas na biblioteca **adc.h**, basta realizar a adequação de cada uma das funções da biblioteca para o padrão do driver. A função que devolve informação **LeValorAD()**, deve retornar o valor através do parâmetro. Na inicialização do driver, é necessário apenas realizar a chamada da função **InicializaAd()** da biblioteca e ajustar as funções internas do driver. O restante do driver apenas segue o padrão definido.

```

1  //drv_adc.h
2  #ifndef DRVADC_H
3  #define DRVADC_H
4
5  enum {ADC_LE_VALOR, ADC_END} func;
6  ptrfunc thisFunctions[ADC_END];
7
8  #endif
9
10 //drv_adc.c
11 char init(void* parameters){
12     drv_id = (char) parameters;
13     InicializaAd();
14     return SUCESS;
15 }
16
17 char LeValor(void* parameters){

```

```
18     int v;
19     v = LeValorAD();
20     *(int*)parameter = v;
21 }
22
23 driver* getDriver(void){
24     thisDriver.Functions = thisFunctions;
25     thisDriver.init = init;
26     thisFunction[ADC_LE_VALOR] = LeValor;
27     return thisDriver;
28 }
```
